



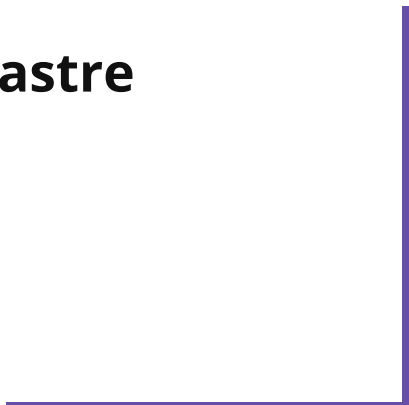
Entrenamiento de Grandes Modelos de Lenguaje

Guillermo Moncecchi - Ignacio Sastre

InCo - Fing - UdelaR

Segunda Escuela Sudamericana de NLP

Agosto de 2026 - Buenos Aires

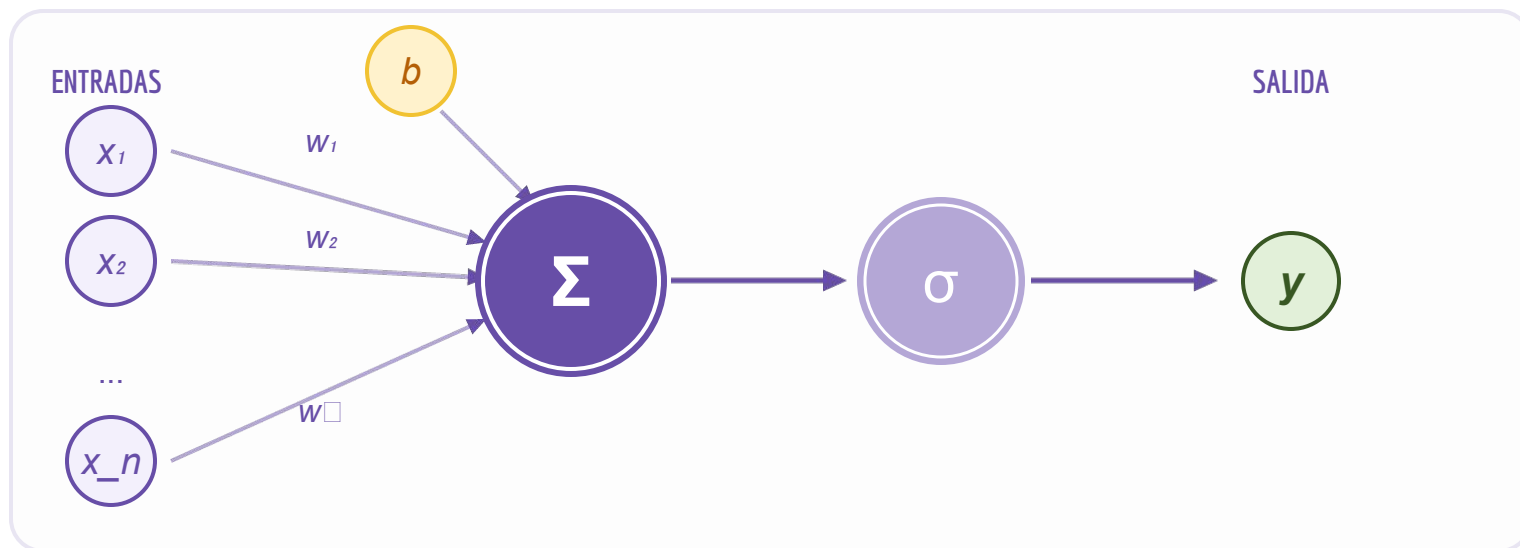




Introducción a las Redes Neuronales



Unidad Sigmoide

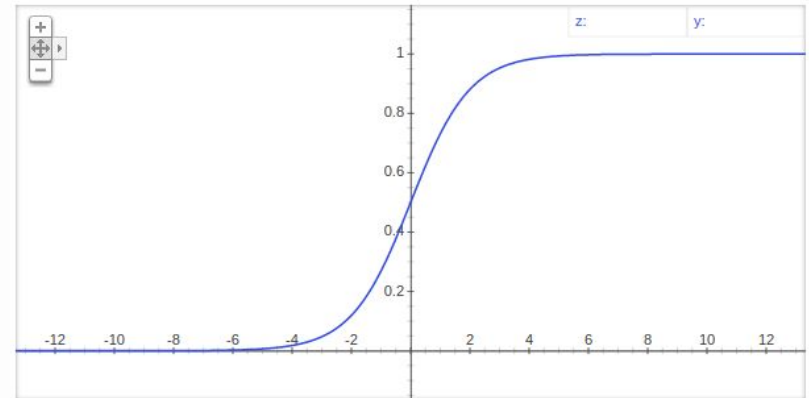


$$y = \sigma\left(\sum_i x_i w_i + b\right)$$

Unidad Sigmoide (a.k.a. Regresión Logística)

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Comportamiento de la Curva



- Para valores altos de z , la salida tiende a **1**.
- Para valores bajos de z , la salida tiende a **0**.
- En $z = 0$, el valor es exactamente **0.5**.

Ejemplo: Análisis de Sentimiento

Las entradas de mi red son números, y la salida también.

¿Cómo hago para representar mi texto mediante números?

P (Positivas): {buenísima, buena, gustó, linda, recomiendo}

N (Negativas): {horrible, divague, mamarracho}

x_1 = cantidad de palabras de la lista P en el texto

x_2 = cantidad de palabras de la lista N en el texto

x_3 = la palabra “no” aparece en el texto

x_4 = el signo de exclamación “!” aparece en el texto

x_5 = cantidad de palabras en el texto

...

Ejemplo: Análisis de Sentimiento

Texto de Entrada

"Escenas traídas de los pelos. No la recomiendo."

Pesos & Predicción

$w = [0.5, 0.6, -0.8, 0.3, -0.1]$

$$\frac{1}{1 + e^{-x \cdot w}} = 0.2497$$

Salida Sigmoide (y)

Características Extraídas

P (Positivas): {buenísima, buena, gustó, linda, recomiendo}

N (Negativas): {horrible, divague, mamarracho}

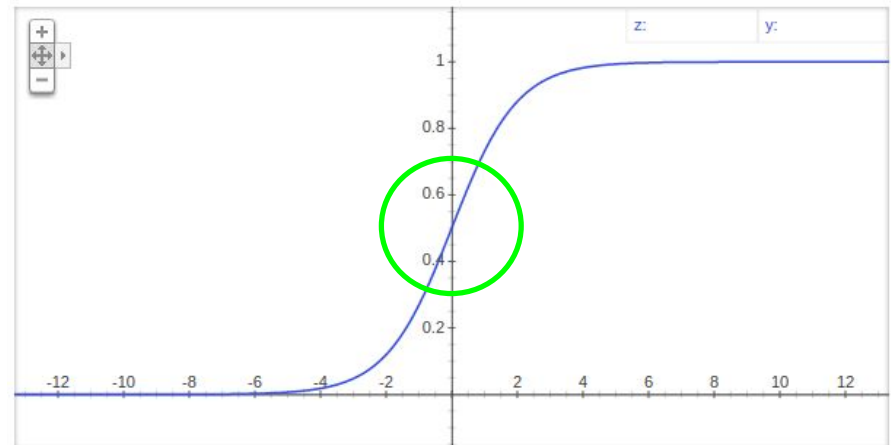
x_1 (palabras de P en texto)= 1

x_2 (palabras de N en texto)= 0

x_3 (palabra "no" en texto)= 1

x_4 (signo "!" en texto)= 0

x_5 (total de palabras)= 8



Ejemplo: Análisis de Sentimiento

?

Texto de Entrada

"Escenas traídas de los pelos. No la recomiendo."

Pesos & Predicción

$w = [0.5, 0.6, -0.8, 0.3, -0.1]$

$$\frac{1}{1 + e^{-x \cdot w}} = 0.2497$$

Salida Sigmoide (y)

Características Extraídas

P (Positivas): {buenísima, buena, gustó, linda, recomiendo}

N (Negativas): {horrible, divague, mamarracho}

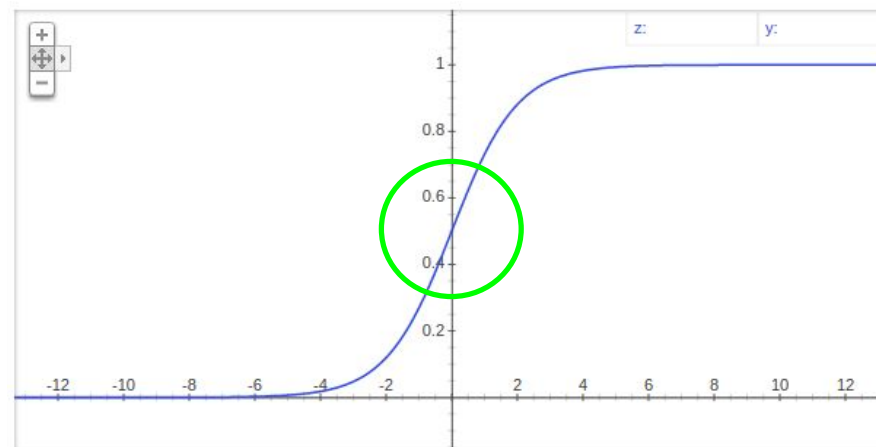
x_1 (palabras de P en texto) = 1

x_2 (palabras de N en texto) = 0

x_3 (palabra "no" en texto) = 1

x_4 (signo "!" en texto) = 0

x_5 (total de palabras) = 8



Entrenamiento

¿Cómo aprendo los mejores pesos w ?

Conjunto de entrenamiento, ejemplos:

$t(1) \rightarrow y(1)$

$t(2) \rightarrow y(2)$

...

$t(m) \rightarrow y(m)$

Siendo cada $t(i)$ un texto, cada $y(i)$ vale 0 o 1

Representación de Entrada

Pero de cada ejemplo $t(i)$ extraigo el vector de features:

$x_1(i), x_2(i), \dots, x_k(i) = x(i)$

Este vector representa las características numéricas extraídas de cada texto para el entrenamiento.

Función de pérdida (Loss)

Objetivo de la Red

Necesito una forma de comparar los valores obtenidos con mi red y los valores esperados.

Para cierto ejemplo x:

$$\hat{\mathbf{y}} = \sigma(\mathbf{x} \cdot \mathbf{w})$$

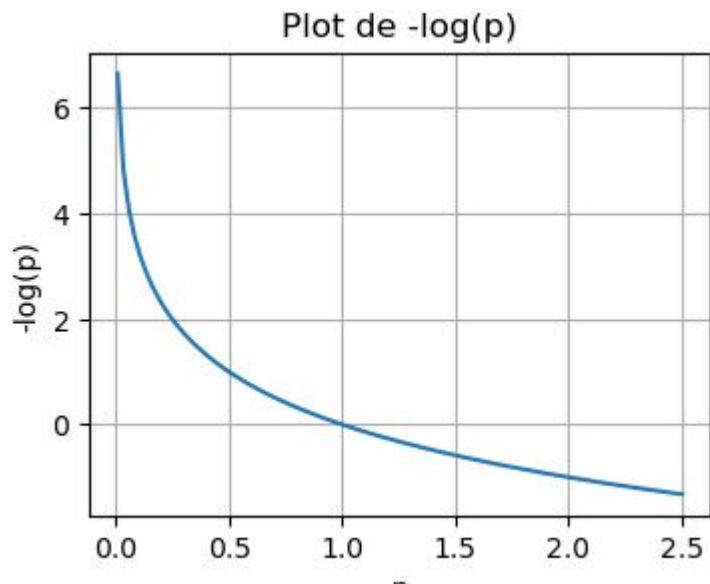
$\mathbf{y}$ es el gold standard (valor real)

Métrica de Error

Para estos casos en que $\mathbf{y}$ puede tomar los valores 0 o 1, se suele usar la **entropía cruzada**:

$$\begin{aligned} L_{CE} &= -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right] \\ &= -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(\sigma(x^{(i)} \cdot w)) + (1 - y^{(i)}) \log(1 - \sigma(x^{(i)} \cdot w)) \right] \end{aligned}$$

Función de pérdida (Loss)



$$H(p) = \sum_{i=1}^n p_i I(p_i) = \sum_{i=1}^n p_i \log(1/p_i) = \sum_{i=1}^n p_i [-\log(p_i)] = - \sum_{i=1}^n p_i \log(p_i)$$

$$H(p, q) = - \sum_{i=1}^n p_i \log(q_i)$$

$$L(y, \hat{y}) = - \sum_{i=1}^m \sum_{j=1}^C y_j^{(i)} \log \hat{y}_j^{(i)}$$

Formalización del problema

Se suele denominar θ al conjunto de parámetros de una familia de funciones de aprendizaje automático (en este caso una red neuronal)

En el ejemplo, nuestro θ es w

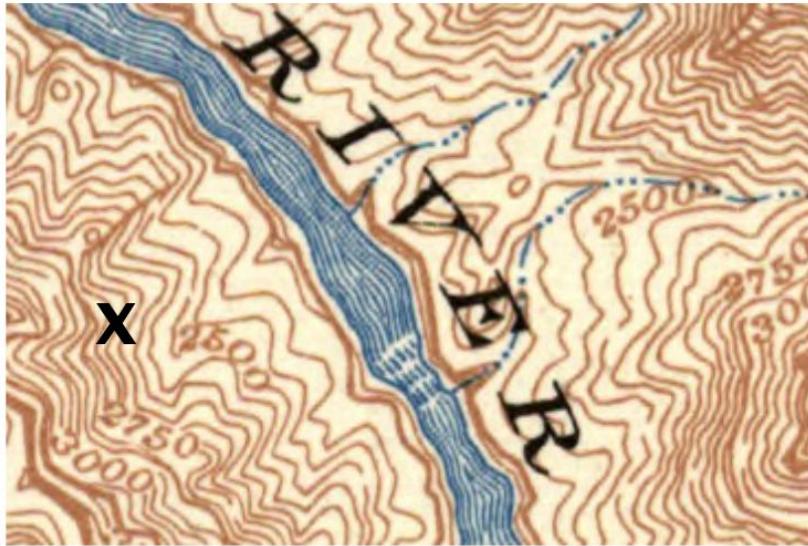
Entonces la función a aprender es $\hat{y} = f(x; \theta)$

Queremos encontrar el conjunto de parámetros que haga mínimo lo siguiente:

$$\hat{\theta} = \operatorname{argmin}_{\theta} \frac{1}{m} \sum_{i=1}^m L_{\text{CE}}(f(x^{(i)}; \theta), y^{(i)})$$

¿Cómo encuentro el mínimo de mi función de pérdida?

¿Cómo llegar al fondo del cañón del río?



01

Observar el entorno

Mirar 360° alrededor de nuestra posición actual.

02

Evaluar la pendiente

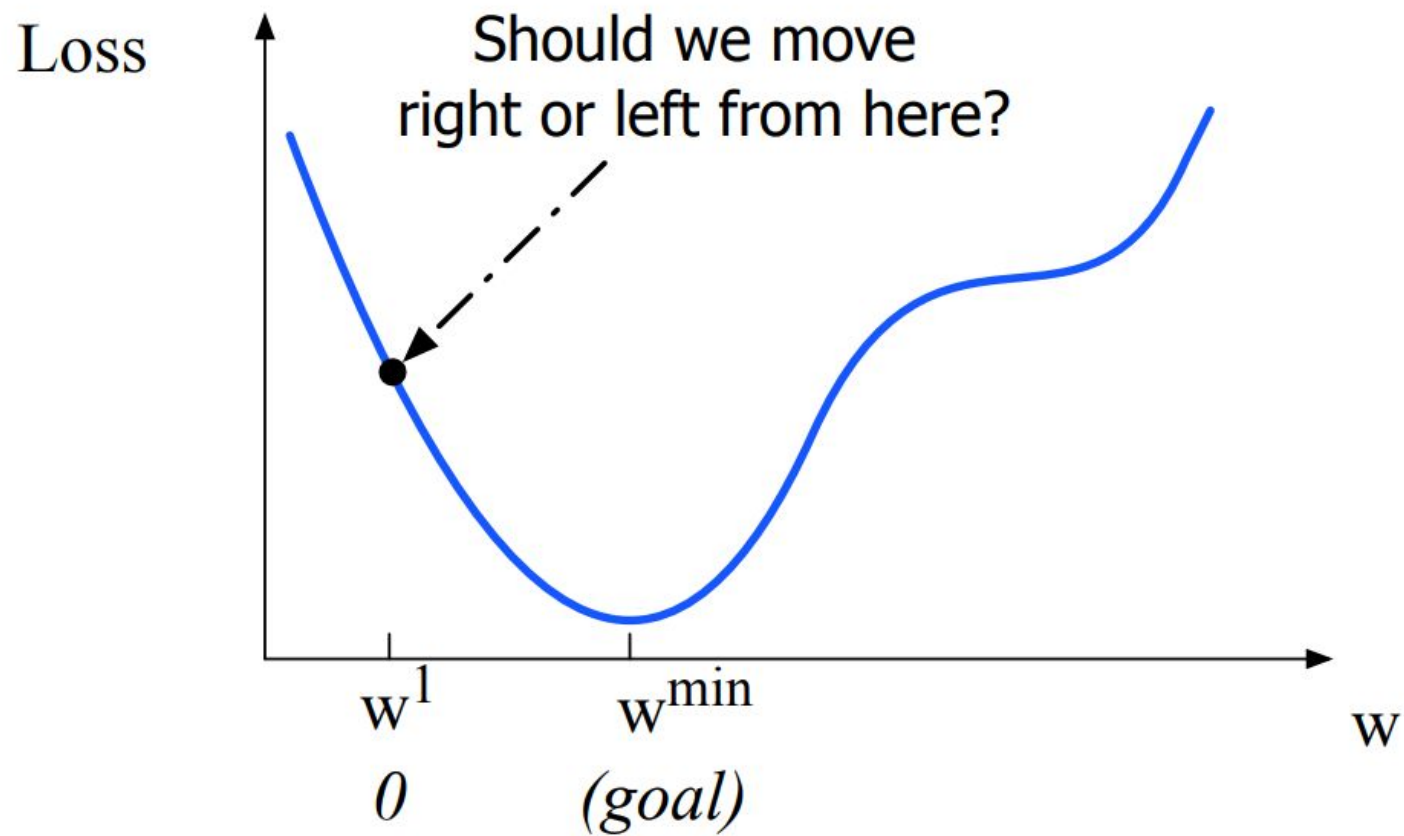
Encontrar la dirección en que la pendiente es más pronunciada hacia abajo.

03

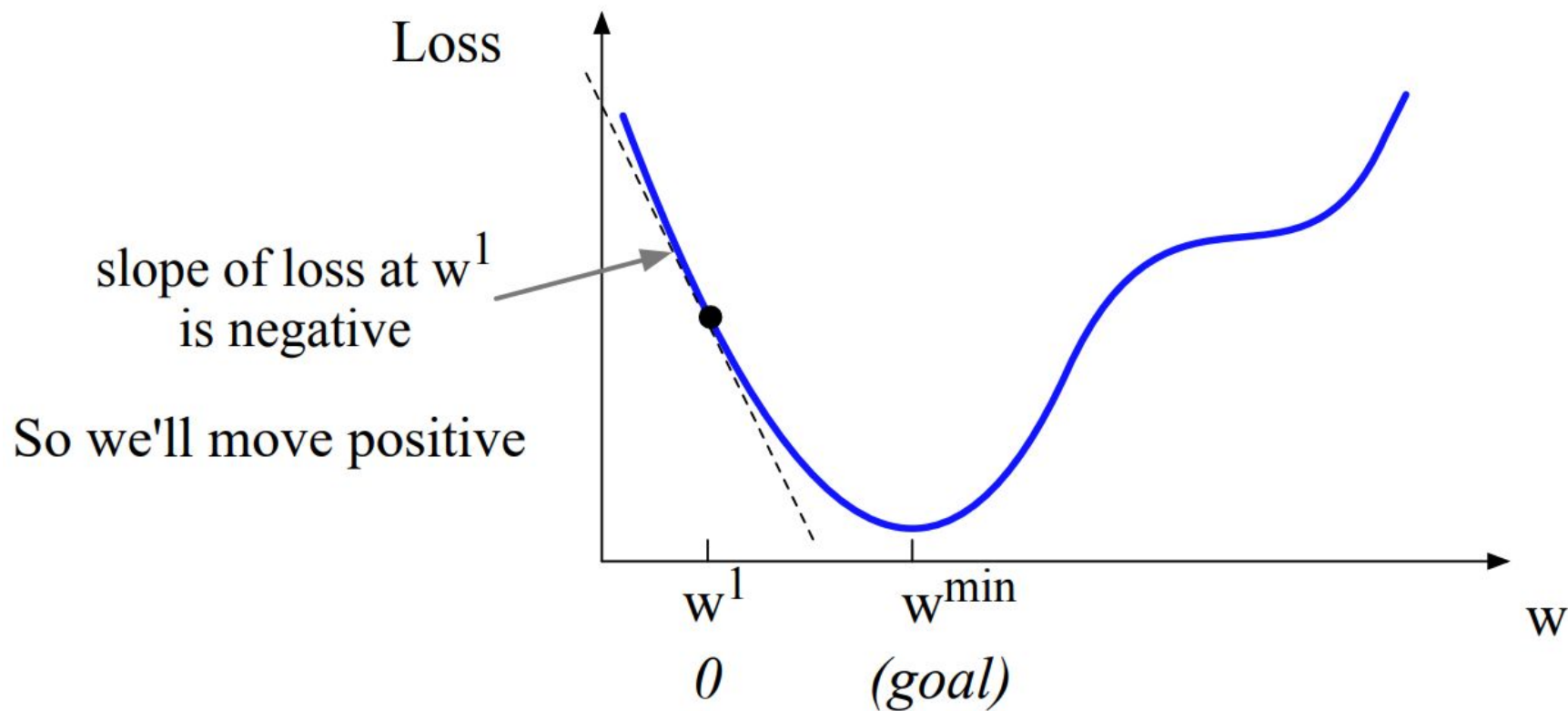
Dar el paso

Caminar con decisión en esa dirección.

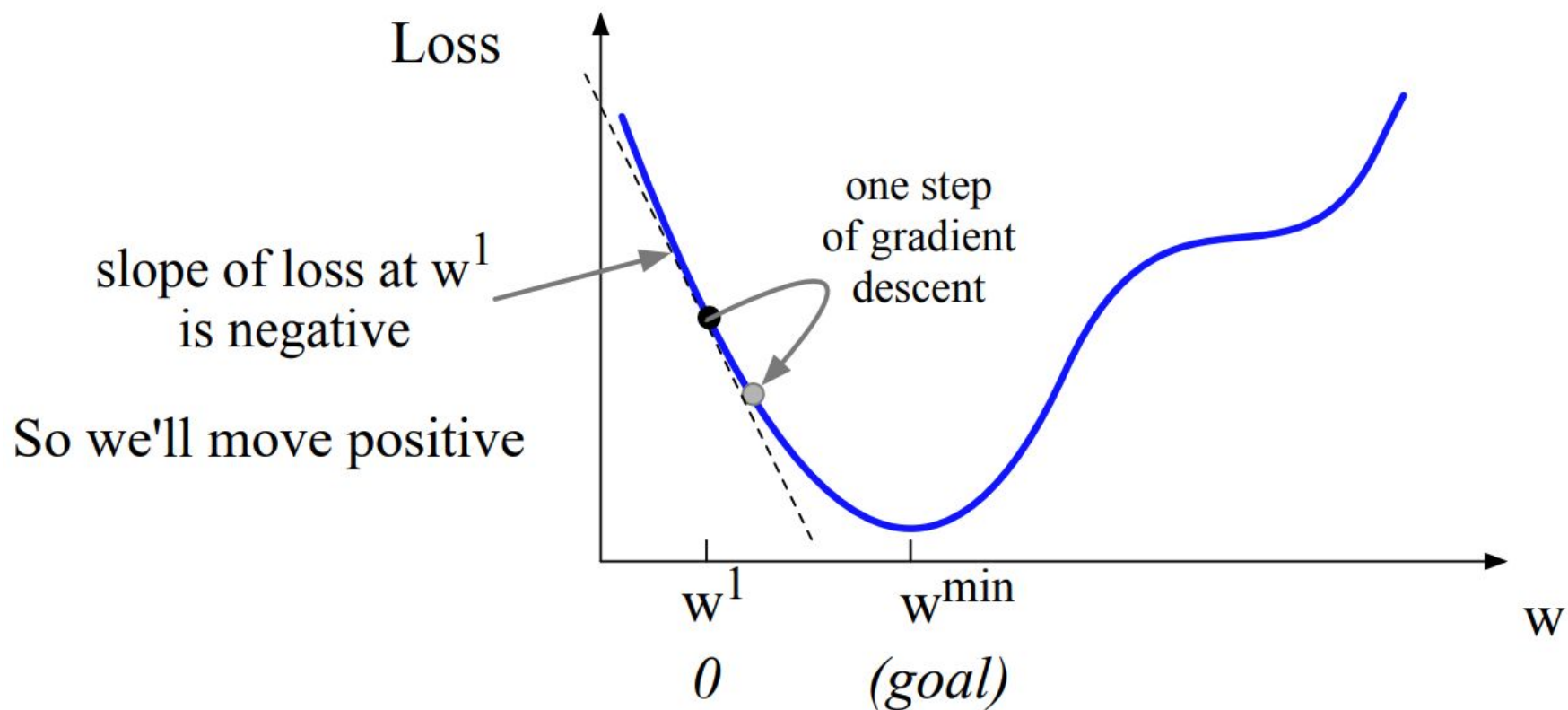
Visualización en una sola dimensión



Visualización en una sola dimensión



Visualización en una sola dimensión



En varias variables: Gradiente

1. El Gradiente

Vector de derivadas parciales que apunta en la dirección de máximo crecimiento de la función.

2. Descenso por Gradiente

Algoritmo de optimización que busca el mínimo de la función de pérdida moviéndose en la dirección opuesta al gradiente.

3. Tasa de Aprendizaje (η)

Hiperparámetro que define el tamaño del paso en cada iteración del proceso de optimización.

Regla de Actualización de Pesos

$$w^{t+1} = w^t - \eta \frac{d}{dw} L(f(x; w), y)$$

⋮

η : Tasa de Aprendizaje

Determina la velocidad de ajuste de los pesos. Un valor muy alto puede causar divergencia; un valor muy bajo hace que converja más lentamente

Descenso por gradiente batch versus estocástico

Tres formas de entrenar:

OPCIÓN 1

SGD

Gradiente Estocástico

Presenta **un único ejemplo a la vez** para calcular el gradiente y realizar la actualización de pesos.

OPCIÓN 2

Batch

Entrenamiento Completo

Coloca **todos los ejemplos** en una matriz y calcula el gradiente para todo el conjunto simultáneamente.

RECOMENDADO

Mini-batches

El punto medio

La alternativa que **mejor funciona**:

- Se elige un lote de ejemplos (ej. 512 o 1024) para actualizar el gradiente.
- Se repite con el siguiente mini-batch hasta completar el conjunto.

Resumiendo

Función de Entrada

Tenemos una función que toma **valores reales** como entradas para iniciar el proceso.

Procesamiento

La función:

- Multiplica cada entrada por un **peso**.
- Suma los resultados y añade un **sesgo**.
- Aplica una **función de activación** (e.g. sigmoide).

Optimización

Para aprender los pesos:

- Calculamos el error comparando con ejemplos conocidos.
- Ajustamos los pesos para minimizar la pérdida usando **descenso por gradiente**.

Unidad sigmoide - Notación

Vector de Entrada

Representación de las características de entrada del modelo mediante un vector de dimensión n .

$$\mathbf{x} \in \mathbb{R}^n = \mathbf{a}^{(1)} = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}$$

Nuestra función de activación

$$h_{\theta}(\mathbf{x}) = \mathbf{a}^{(2)} \in \mathbb{R} = g(w_1^{(2)} \cdot a_1^{(1)} + w_2^{(2)} \cdot a_2^{(1)} + \dots + w_n^{(2)} \cdot a_n^{(1)} + b^{(2)})$$

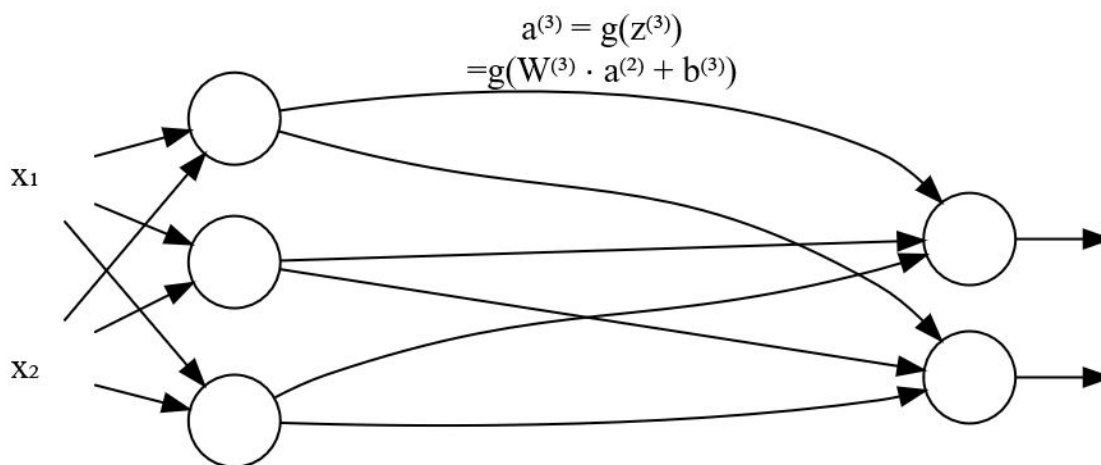
$$h_{\theta}(\mathbf{x}) = \mathbf{a}^{(2)} \in \mathbb{R} = g(z^{(2)}) = g(w^{(2)} \cdot \mathbf{x} + b^{(2)})$$

Siendo $\mathbf{w}^{(2)}$ los parámetros de la capa 2, $\mathbf{b}^{(2)}$ un término de sesgo y $g(\mathbf{z})$ la función de activación.

Redes neuronales - Notación

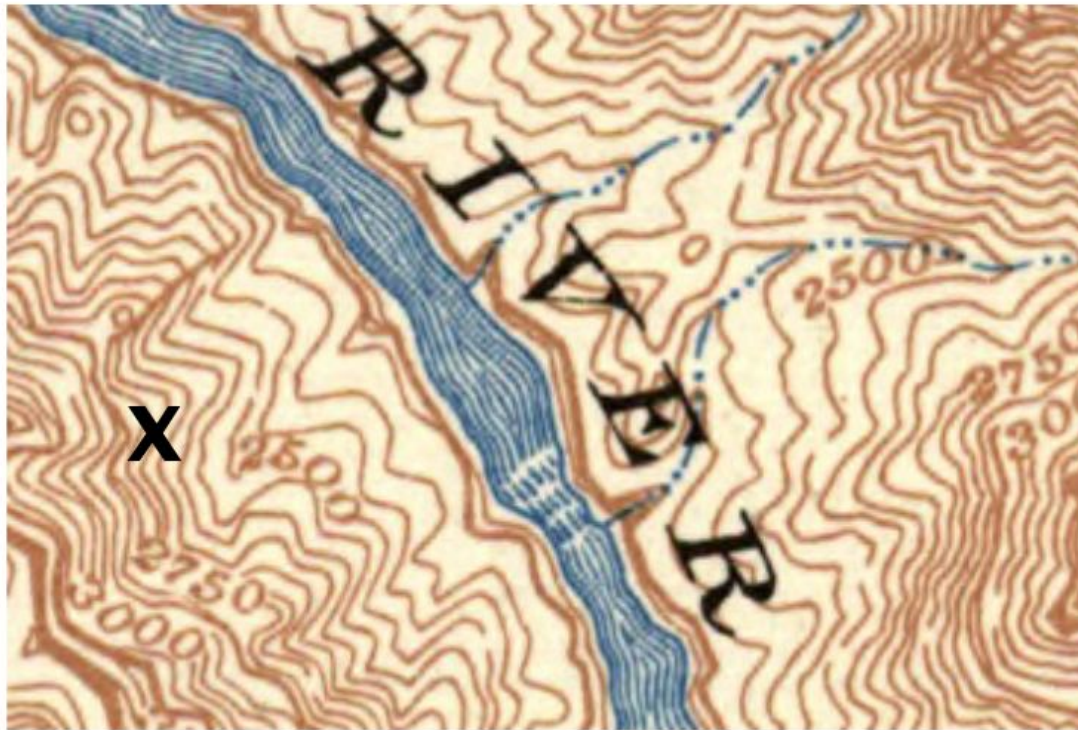
Generalizando el caso anterior, podemos denotar cualquier red neuronal:

$$a^{(1)} \in \mathbb{R}^n = x = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}, \quad a^{(j)} \in \mathbb{R}^{s_j} = \begin{bmatrix} a_1^{(j)} \\ \vdots \\ a_{s_j}^{(j)} \end{bmatrix} = g(W^{(j)} \cdot a^{(j-1)} + b^{(j)})$$



Redes neuronales - Aprendizaje

¿Cómo aprendemos los pesos? Descenso por gradiente!



Redes neuronales - Aprendizaje

Necesitamos una función de pérdida. Las funciones de pérdida tienen algunas características:

01

Solamente dependen de la última capa de la red

02

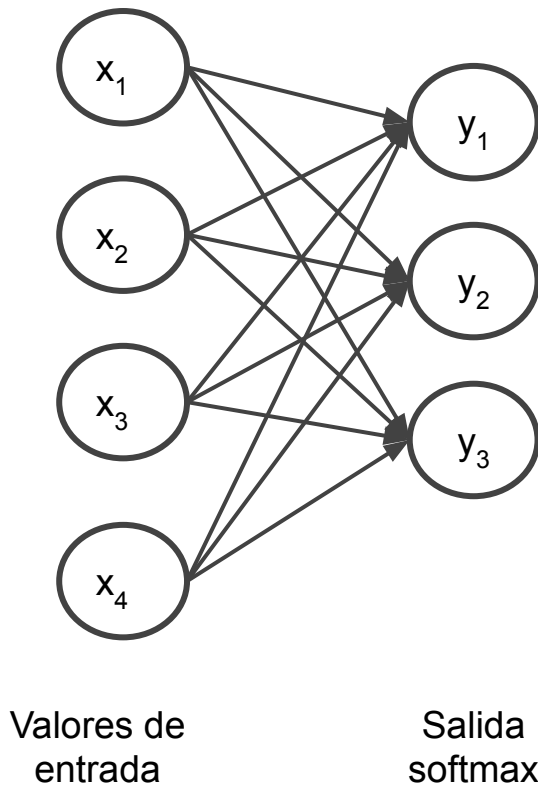
Pueden calcularse como el promedio de la pérdida de cada ejemplo

03

Por supuesto, son menores cuanto más parecida es la salida a cada ejemplo

Cuál usar va a depender de la salida de nuestra red

Redes Neuronales - Salida multiclase



Clasificación de más de dos clases

Queremos que la salida de la red sea una distribución de probabilidad.

Función de activación *softmax*

$$P(j|x) = \frac{e^{y_j}}{\sum_k e^{y_k}}$$

Normalización

Los valores de los y_i corresponden a una distribución de probabilidad.

Clasificación Multiclase

¿Cómo queda la función de pérdida?

Definición del escenario

Si tenemos m ejemplos, cada uno con una categoría entre k clases.

Para cada ejemplo y_i , únicamente una de las clases es la correcta.

Es decir que, en cada caso:
 $y_{ij} = 1$, y los demás son 0

Función de Pérdida

La entropía cruzada en este caso se define como:

$$L_{CE} = -\frac{1}{N} \sum_{i=1, y_{ij}=1}^N \log(\hat{y}_{ij})$$

Redes Neuronales: Backpropagation

El Problema

Las capas apiladas son composiciones de funciones. Calcular el gradiente implicaría "desplegar" toda la función e ir derivando respecto a cada parámetro, lo cual es computacionalmente muy costoso.

La Intuición de Backpropagation

La idea central es explotar la arquitectura de la red para partir "de atrás hacia adelante", reutilizando de forma eficiente los resultados intermedios (esencialmente, utilizando la regla de la cadena)

Dos Pasadas del Algoritmo

Para computar el gradiente, el algoritmo realiza de manera estructurada dos recorridos fundamentales por la red:

1. Forward Pass

Se ejecuta la red hacia adelante para una entrada, calculando y guardando las salidas intermedias.

2. Backward Pass

Se propaga el gradiente del error desde la capa de salida hacia la capa de entrada, reutilizando los valores previos.

Backpropagation

Grafo de Computación

La diferenciación se puede hacer de forma automática teniendo un grafo de computación.

Las operaciones tensoriales se computan más rápido en hardware específico (Ej. GPUs, TPUs).

Seguimos utilizando **SGD con mini-batches**.

Términos a tener en cuenta

Step

Un episodio de actualización del gradiente:

- Aplicar la red a un mini-batch
- Realizar backpropagation y actualizar los pesos

Epoch

Pasar una vez por el corpus de entrenamiento entero:

- Generalmente implica varios steps

Resumiendo

Entradas

Tenemos una función que toma valores reales como entradas.

Procesamiento

Esta función multiplica cada entrada por un peso, suma los resultados, le agrega un sesgo, y le aplica una función de activación (e.g. una sigmoide), para obtener el resultado...

muchas veces, y haciendo que cada capa tome como entrada la salida de la anterior

Aprendizaje

Para aprender los pesos, calculamos cuánto se equivoca la función comparando con ejemplos que conocemos, y ajustamos los pesos para que ese error (o pérdida) sea mínimo, utilizando descenso por gradiente

(y usando backpropagation para obtener el gradiente)

Sobreajuste

Las redes neuronales son una familia de funciones con mucha flexibilidad y capacidad de representación.

Podemos representar casi cualquier función matemática con una sola capa oculta¹.

 **¡Por consiguiente tienden a sobreajustar muy fácilmente!**

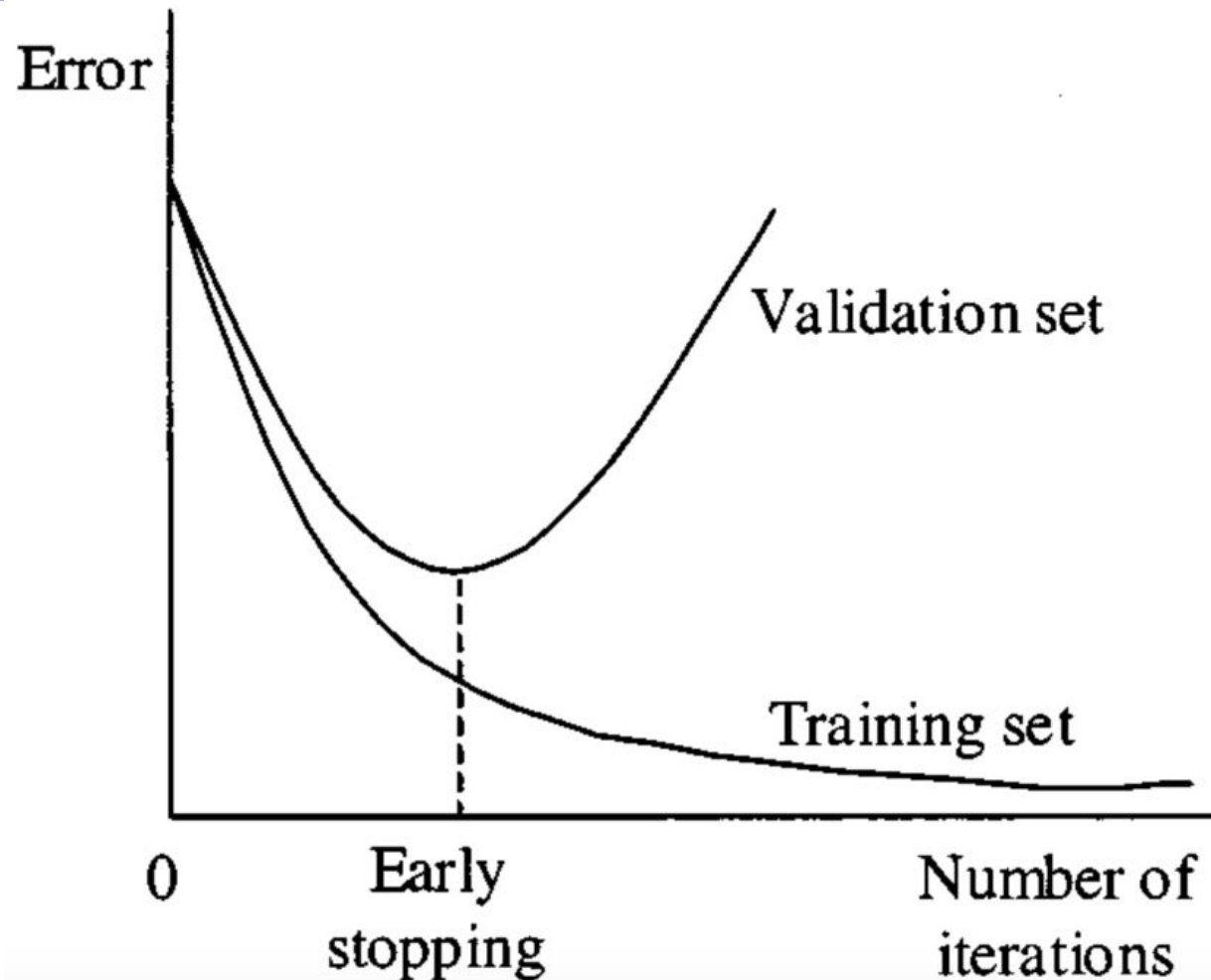
Técnicas de Mitigación

Para combatir el sobreajuste, se implementan diversas estrategias de regularización:

- 1. Regularización** Penalización para restringir la complejidad de los pesos de la red.
- 2. Dropout** Desactivación aleatoria de neuronas durante el proceso de entrenamiento.
- 3. Early Stopping** Interrupción del entrenamiento al detectar que el error de validación deja de mejorar.

¹ *Approximation by Superpositions of a Sigmoidal Function (Cybenko 1989)*

Early Stopping



Hiperparámetros

Hiperparámetros Comunes

- Learning rate
- Tamaño del mini-batch
- Cantidad de capas
- Cantidad de unidades por capa
- Función de activación de cada capa
- Probabilidad de dropout
- Tolerancia de Early stopping

Consideraciones Prácticas

... ¡cada vez habrá más hiperparámetros a optimizar!

Siempre terminamos probando varios modelos para encontrar la mejor configuración.

 **¡Importante! Partir siempre el conjunto de datos en train, dev y test.**

Aprendizaje de Representaciones

Las redes neuronales tienen la capacidad de aprender representaciones intermedias en las capas ocultas a partir de los datos. Estas representaciones (atributos), que no aparecieron en forma de atributos explícitos en la entrada, son sin embargo aprendidas y capturan propiedades de las instancias de entrada. Mostraremos un ejemplo muy sencillo, tomado del libro de Tom Mitchell.

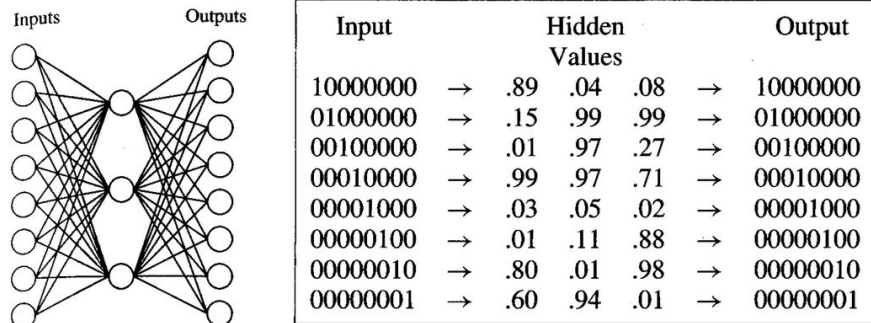


FIGURE 4.7

Learned Hidden Layer Representation. This $8 \times 3 \times 8$ network was trained to learn the identity function, using the eight training examples shown. After 5000 training epochs, the three hidden unit values encode the eight distinct inputs using the encoding shown on the right. Notice if the encoded values are rounded to zero or one, the result is the standard binary encoding for eight distinct values.



Arquitecturas Encoder-Decoder y Transformers



Arquitectura Encoder-Decoder

Una red compuesta por dos subredes:

Encoder

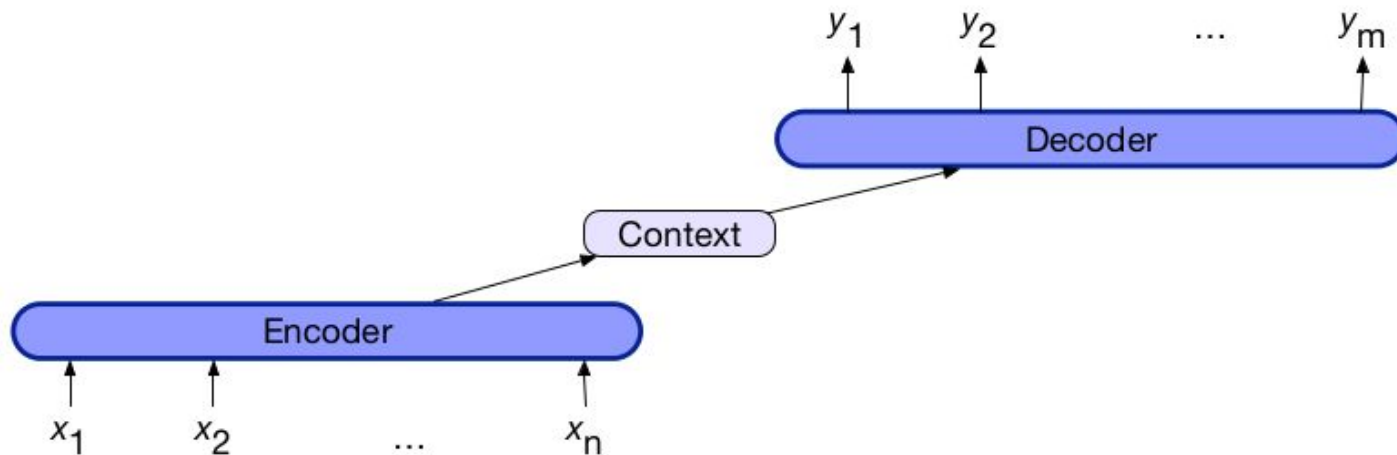
Red que codifica la entrada

Decoder

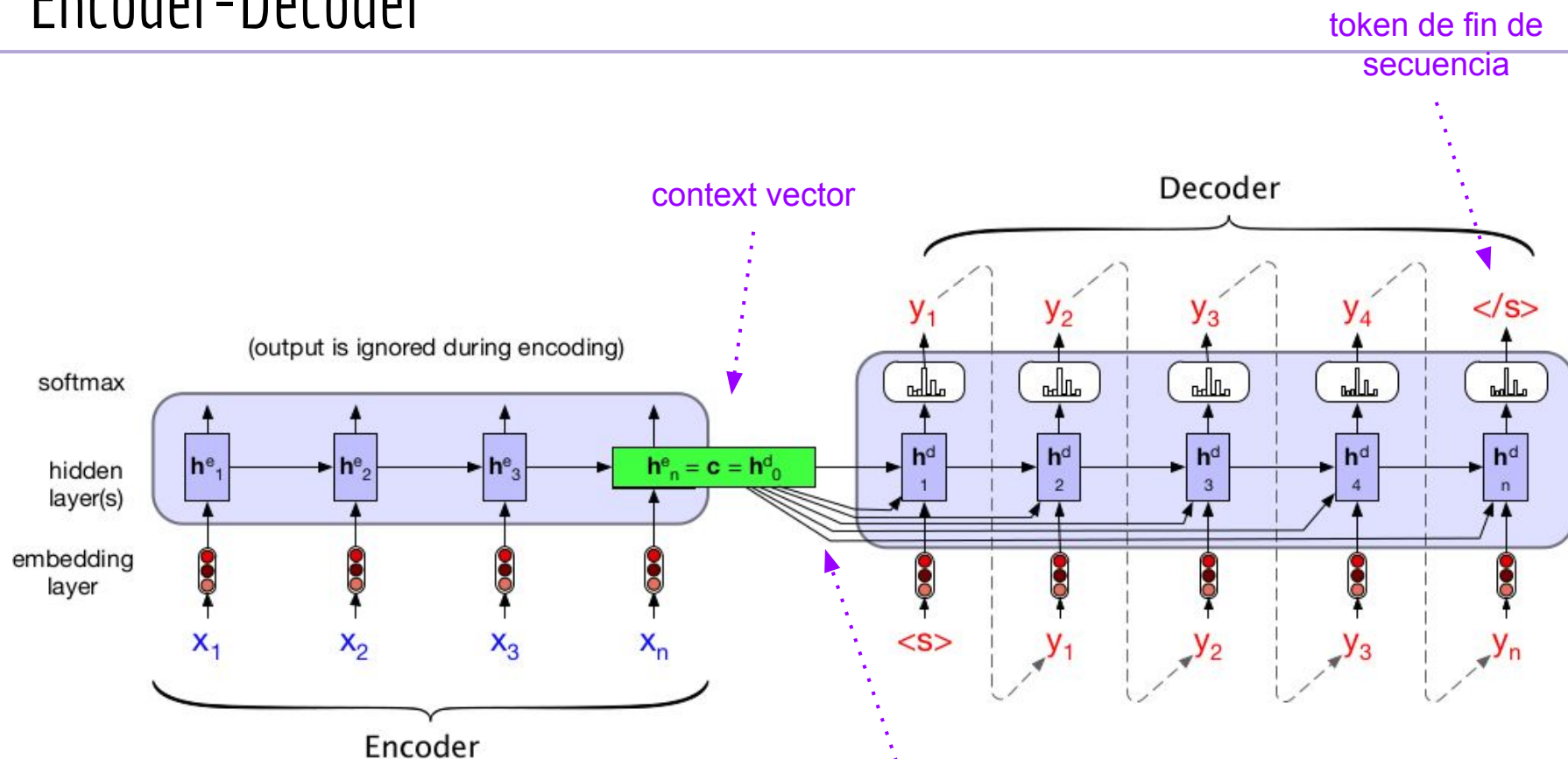
Red que decodifica (y construye) la salida

Context vector

“Embedding” de toda la secuencia de entrada



Encoder-Decoder



debido a que la influencia de c sobre la decodificación de la secuencia puede ir disminuyendo, se suele incluir en cada paso

Encoder-Decoder: entrenamiento

1. Datos de Entrada

Se entrenan con pares de secuencias (entrada – salida).

Por ejemplo, corpus paralelos en traducción automática: oraciones origen en inglés y destino en español.

2. Flujo de Red

La red **encoder** codifica la entrada, se ignoran las salidas intermedias y se mantiene solo el vector de contexto.

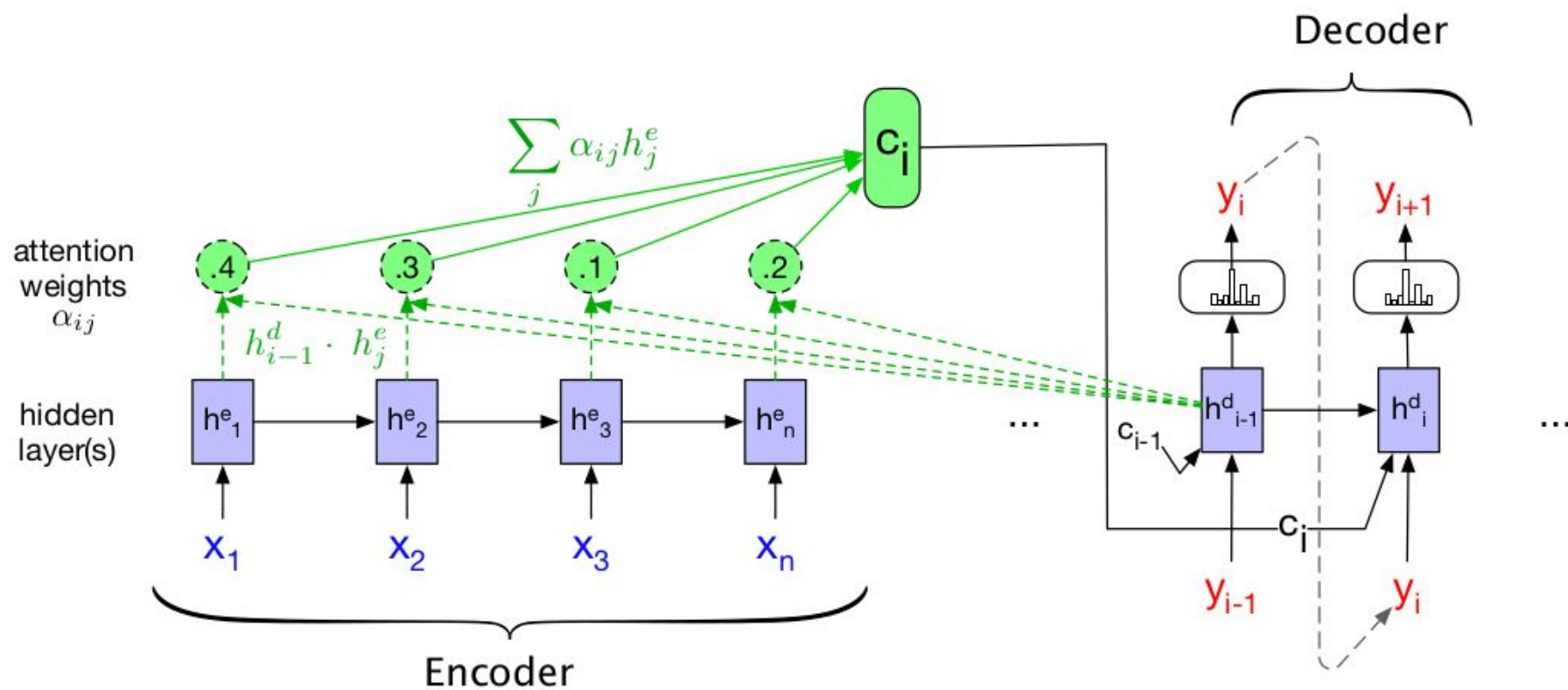
A partir del vector de contexto se decodifica la salida con el **decoder** de forma autorregresiva.

3. Optimización

Se entrena **end-to-end**, minimizando el **loss** de acertar a las palabras correctas.

Se utiliza **teacher forcing** al entrenar la decodificación de la salida.

Mecanismo Atencional



Transformer

Limitación de las RNNs

Las RNNs obligan a procesar una palabra después de otra en orden, creando un cuello de botella secuencial.

Transformer

Libera esta restricción procesando todo en paralelo, permitiendo entrenar con volúmenes masivos de datos.

Mayor velocidad y mejor captura de dependencias a larga distancia.

1. Self-attention / Autoatención

Relaciona cada palabra con el resto.

2. Conexiones residuales

Facilitan el flujo de gradientes en arquitecturas profundas.

3. Embeddings posicionales

Injectan la información del orden que el paralelo omite.

4. Masked attention y Cross-attention

Controlan y dirigen el flujo de información en la red.

Motivación

Los transformers son capaces de crear **embeddings contextuales** (no son los únicos):

Ejemplo (Inglés)

*"The chicken didn't cross the road because **it** was too tired."*

*"The chicken didn't cross the road because **it** was too wide."*

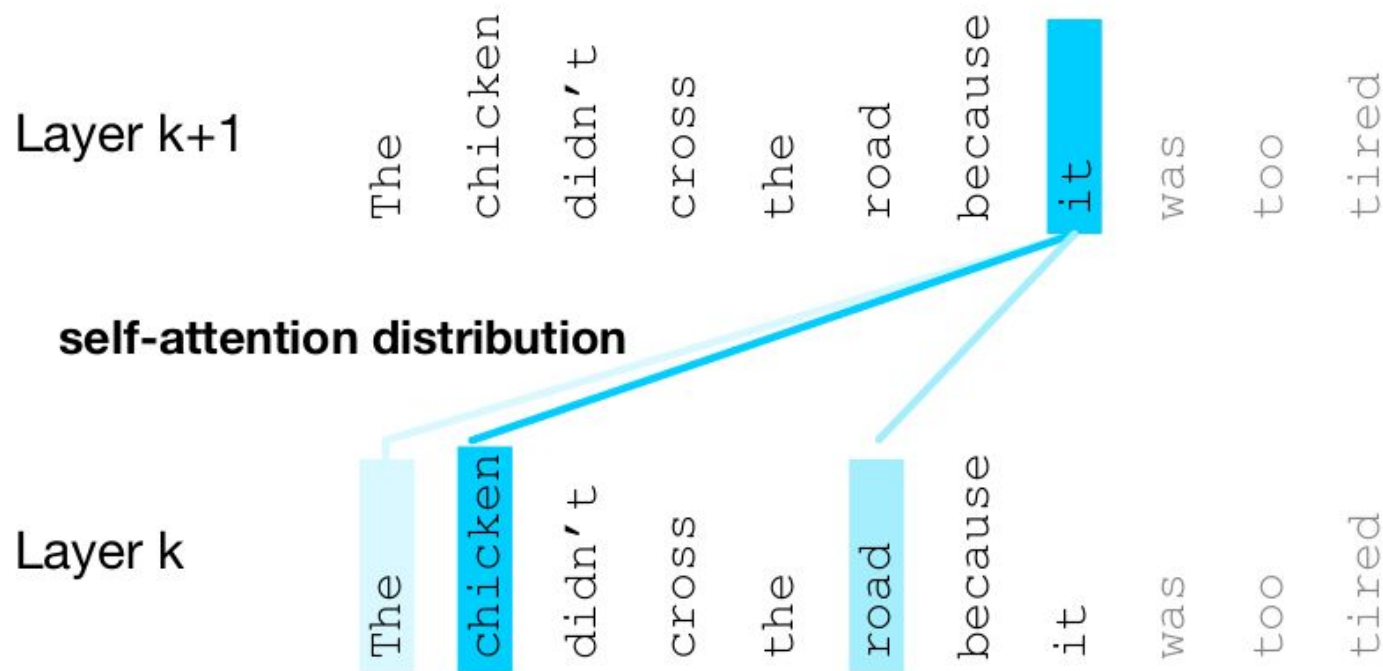
En Español

*"El pollo no cruzó la calle porque **estaba** muy cansado."*

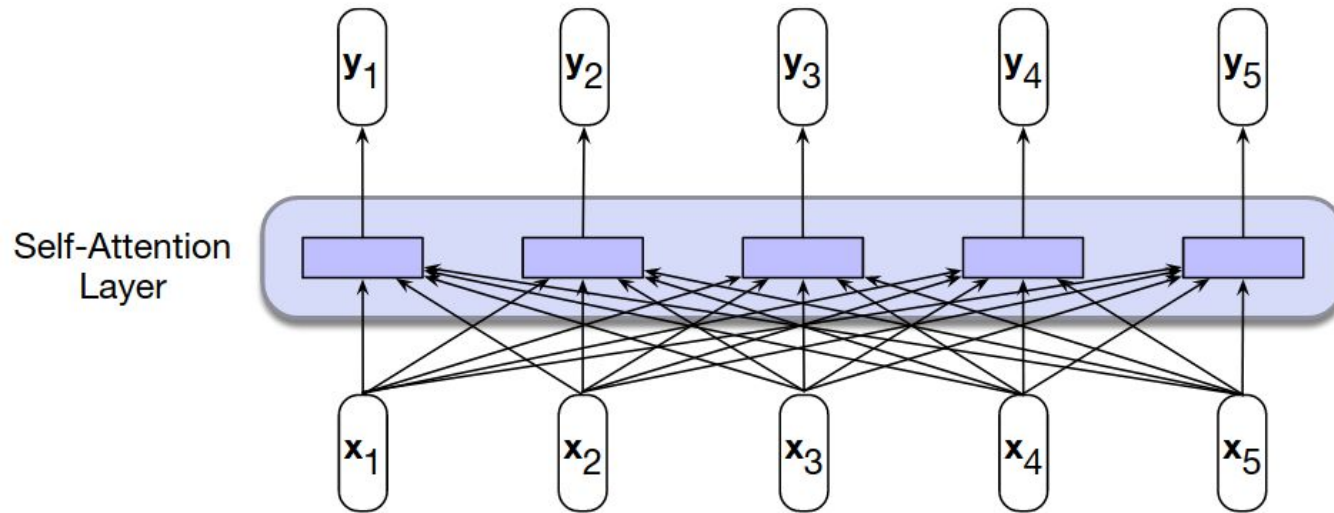
*"El pollo no cruzó la calle porque **estaba** cortada."*

Motivación

¿Qué esperamos que pase con el mecanismo de atención en estos casos?

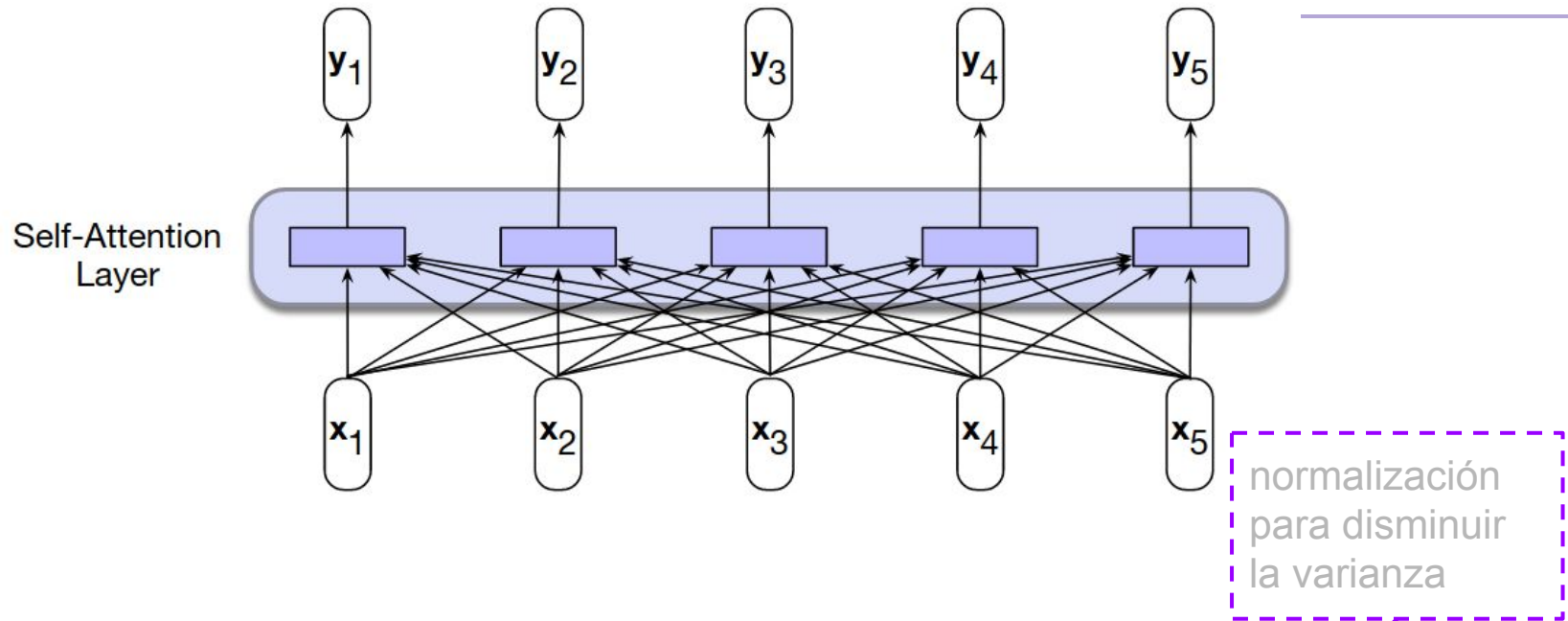


Self-attention



$$y_i = \sum \alpha_{ij} x_j$$
$$\alpha_{ij} = \frac{\text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j))}{\sum_{k=1}^i \exp(\text{score}(\mathbf{x}_i, \mathbf{x}_k))}$$
$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = ?$$

Self-attention



Tres proyecciones de cada x_i (**query**, **key** y **value**), con tres matrices

Los vectores q_i y k_j computan el **score** de cómo influye la entrada j para la entrada i

Luego softmax (en j) que ponderan los v_i

$$q_i = x_i W^Q; \quad k_j = x_j W^K; \quad v_j = x_j W^V$$

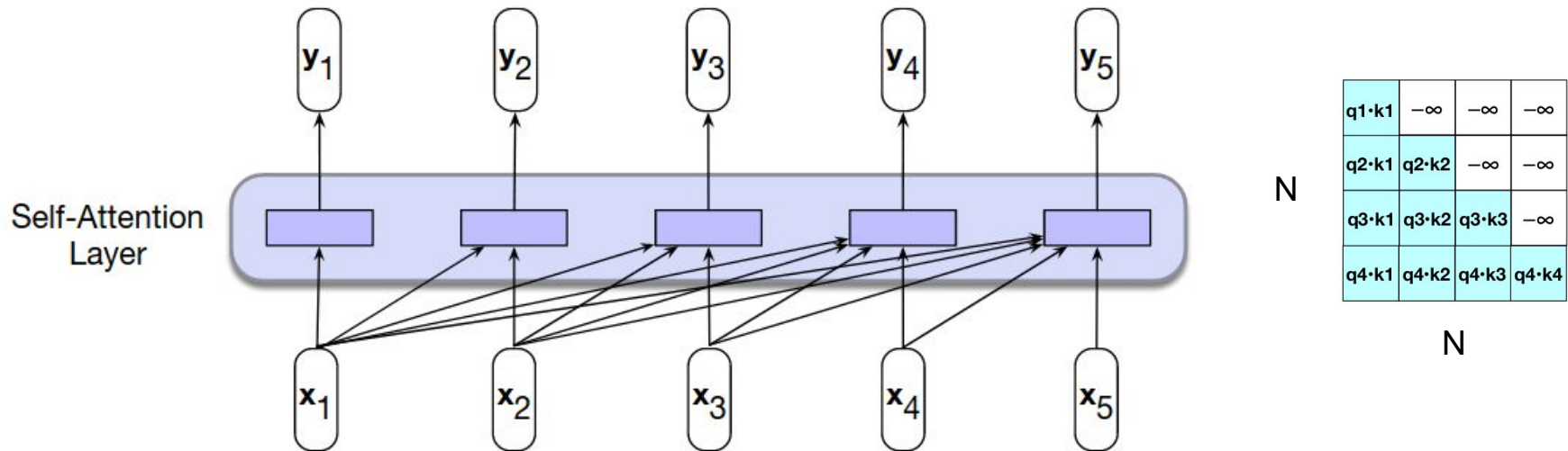
$$\text{score}(x_i, x_j) = \frac{q_i \cdot k_j}{\sqrt{d_k}}$$

$$\alpha_{ij} = \text{softmax}(\text{score}(x_i, x_j))$$

$$y_i = \sum \alpha_{ij} v_j$$

Máscara causal de Self-attention

El mecanismo es igual, pero agrega un enmascaramiento para no ver palabras futuras.



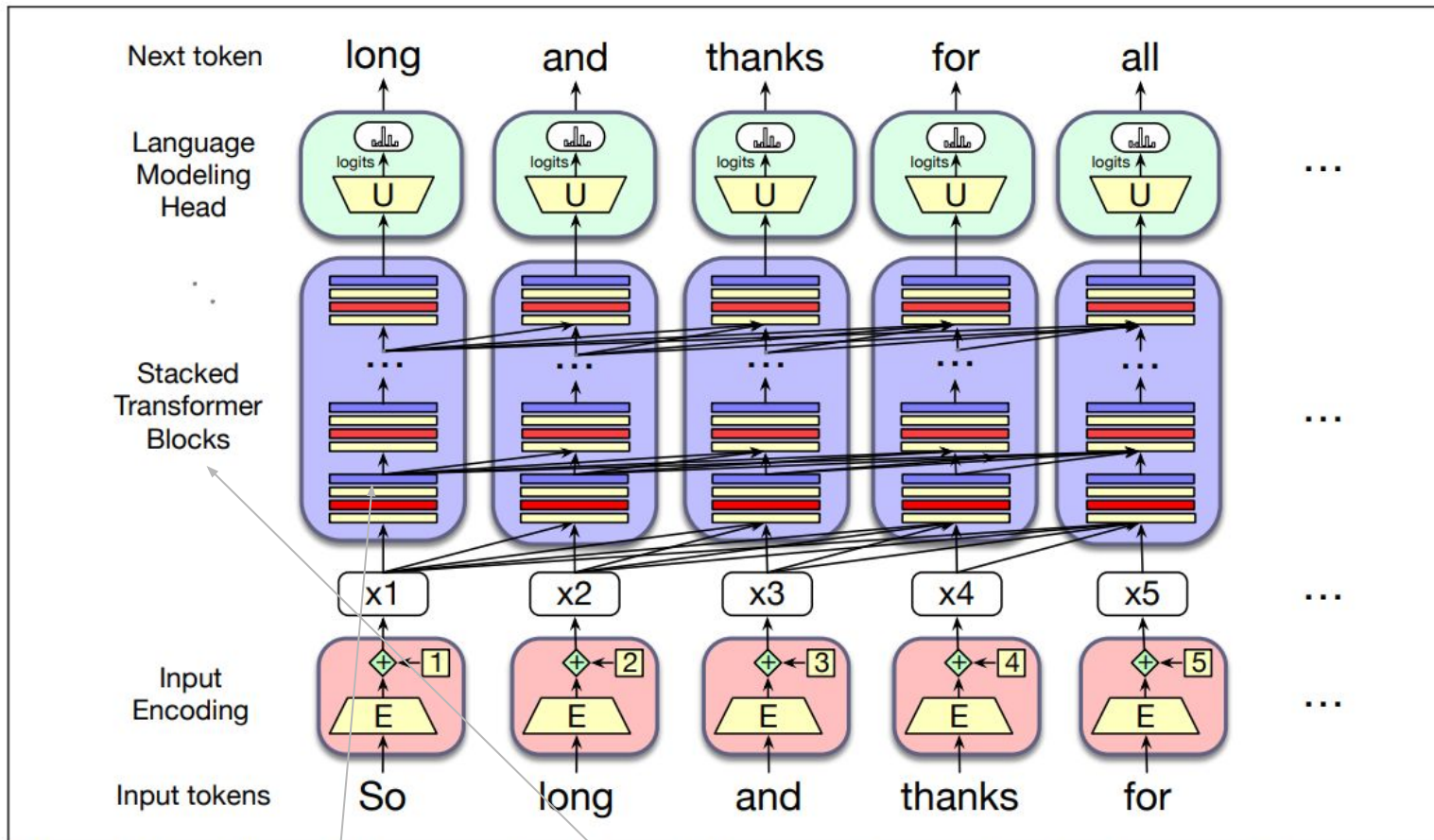


Figure 8.1 The architecture of a (left-to-right) transformer, showing how each input token get encoded, passed through a set of stacked transformer blocks, and then a language model head that predicts the next token.

Mecanismo de
self-attention

Bloque Transformer

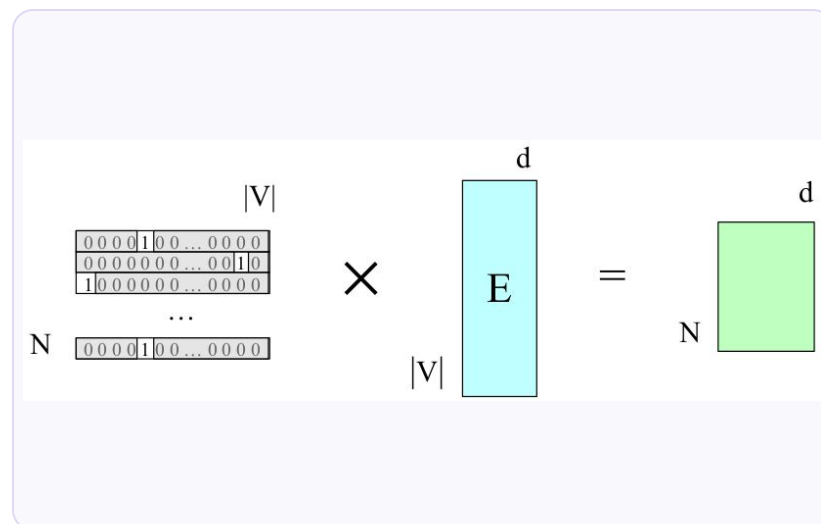
Codificación de la entrada

1. Tokenización

La representación del texto de entrada es mediante tokens (BPE, word-piece, otros...).

2. Embeddings de Entrada

Al entrar a la red, esos tokens pasan por una capa de embeddings para obtener sus representaciones vectoriales.



Pero tenemos un problema extra...

¿Qué pasa con el orden?

Codificación de la entrada

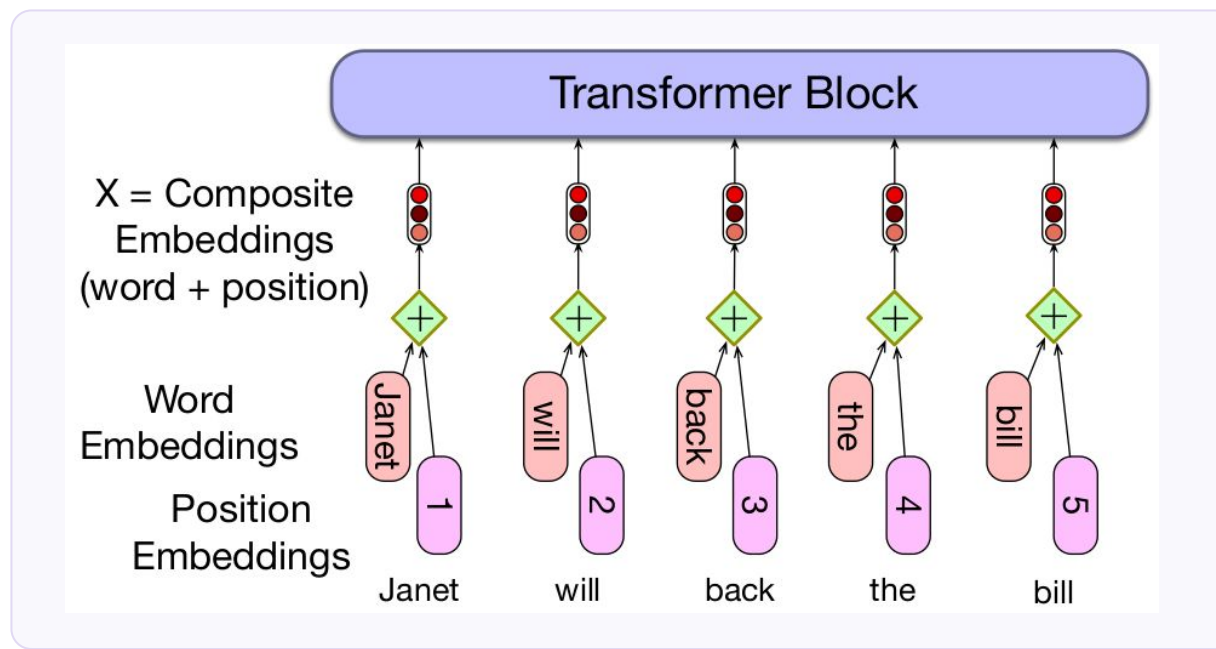
Hasta este punto **no hay noción de orden** en la entrada.

Ni hay una recurrencia que “imponga” orden procesando de a uno.

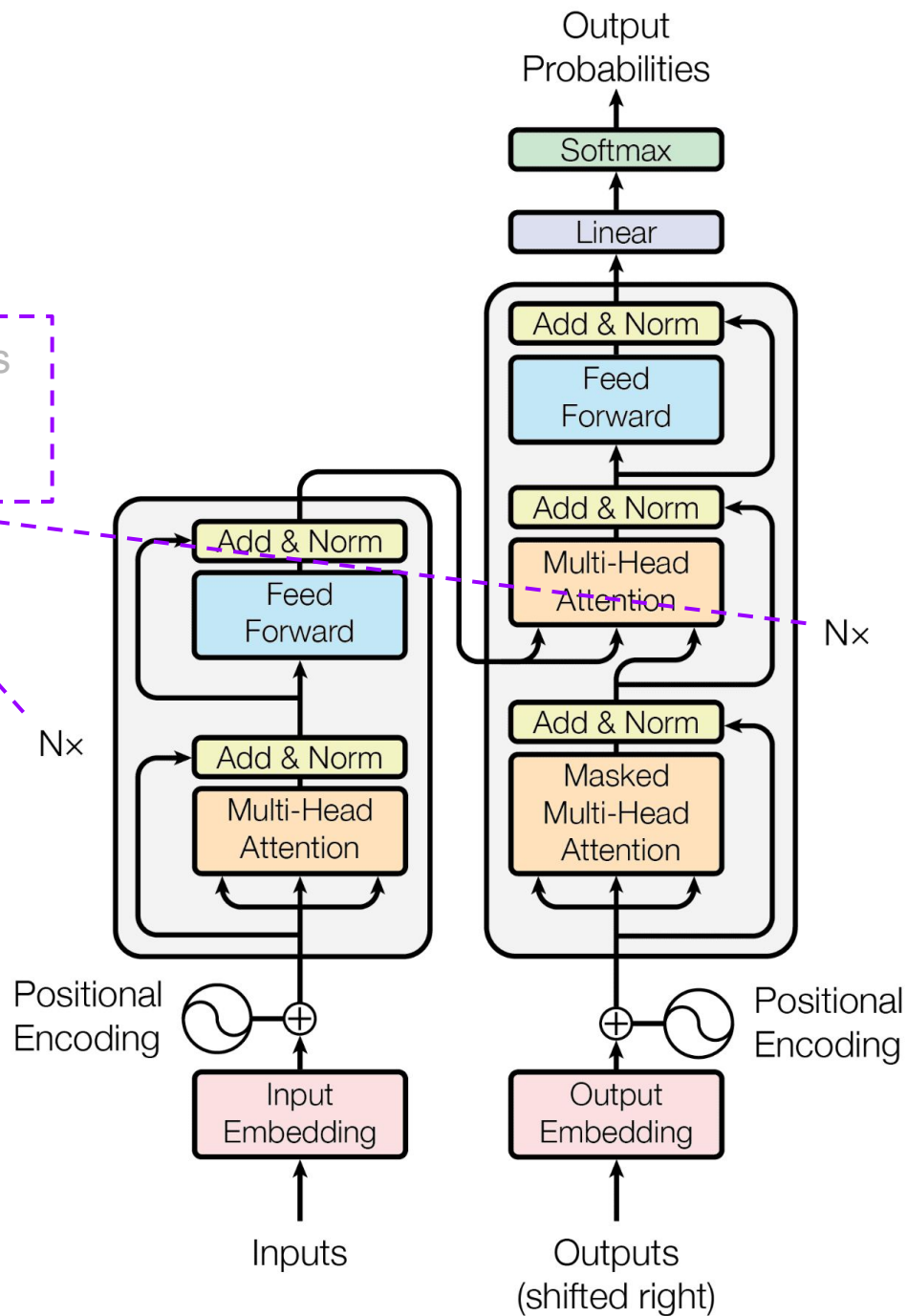
Solución para los Transformers:

“Aumentar” cada x_i para que incluya una representación de su posición.

→ **Embeddings posicionales**



Notar que son varios bloques transformer en stack



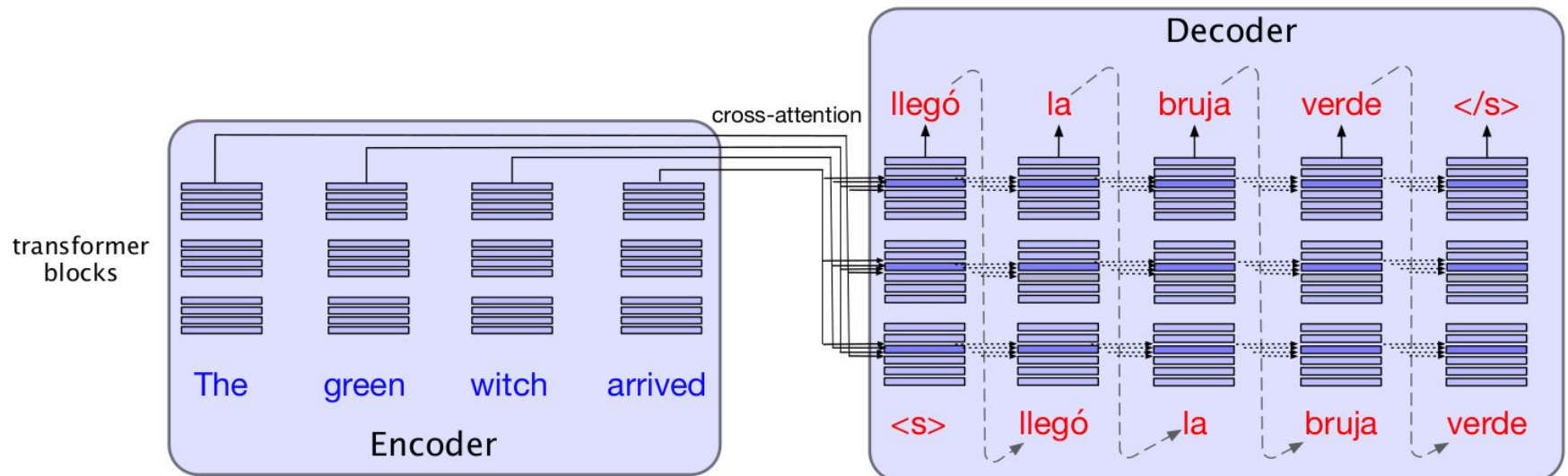
Transformer Encoder-Decoder

Encoder

El encoder crea una representación para cada token

Decoder

- Al decoder lo inicializamos con un token <S>
- Usa masked-attention para codificarlo
- Y cross-attention para combinar ese token con el resultado del encoder
- Lo cual produce como resultado el primer token de salida...
- ...y repetimos el proceso para decodificar el resto



Referencias

Jurafsky, D., & Martin, J. H. (2026). *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition, with Language Models* (3rd edn).

Raschka, S., Liu, Y., Mirjalili, V., & Dzhulgakov, D. (2022). *Machine Learning with PyTorch and Scikit-Learn: Develop Machine Learning and Deep Learning Models with Python*.

Raschka, S. (2024). *Build a Large Language Model (From Scratch)*.

3RA ESCUELA CICADA

Un evento de Khipu x Punta del Este

tokenpal%za

MODELOS DE LENGUAJE



Del **1** al **6**
de noviembre



Punta del
Este



**INSCRIPCIONES
ABIERTAS**

Hasta el 15 de agosto

bit.ly/tercera-escuela



CICADA



UNIVERSIDAD
DE LA REPÚBLICA
URUGUAY

APOYA:



CMAT

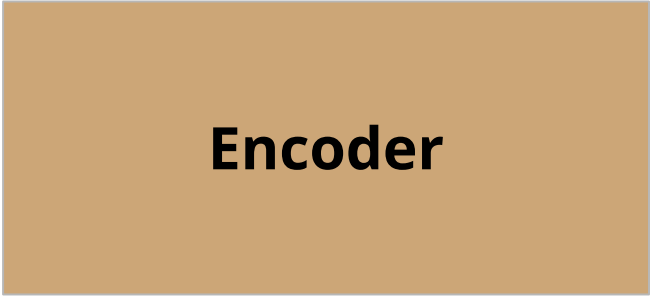


Ahora sí lo divertido...

Construyamos y entrenemos un encoder-decoder basado en Transformers...



Usando el modelo...

A solid tan rectangular block representing the Encoder component of a model.

Encoder

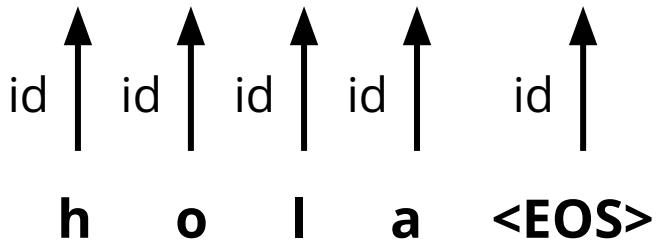
A solid tan rectangular block representing the Decoder component of a model.

Decoder

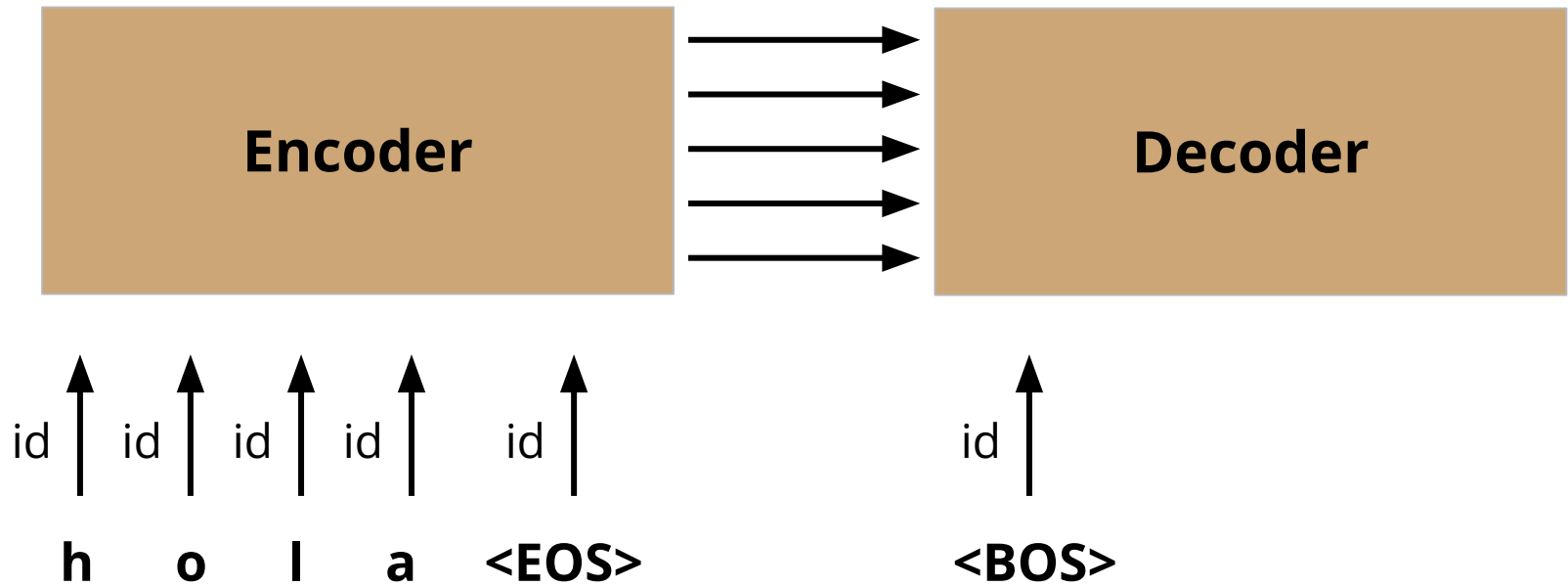
Usando el modelo...

Encoder

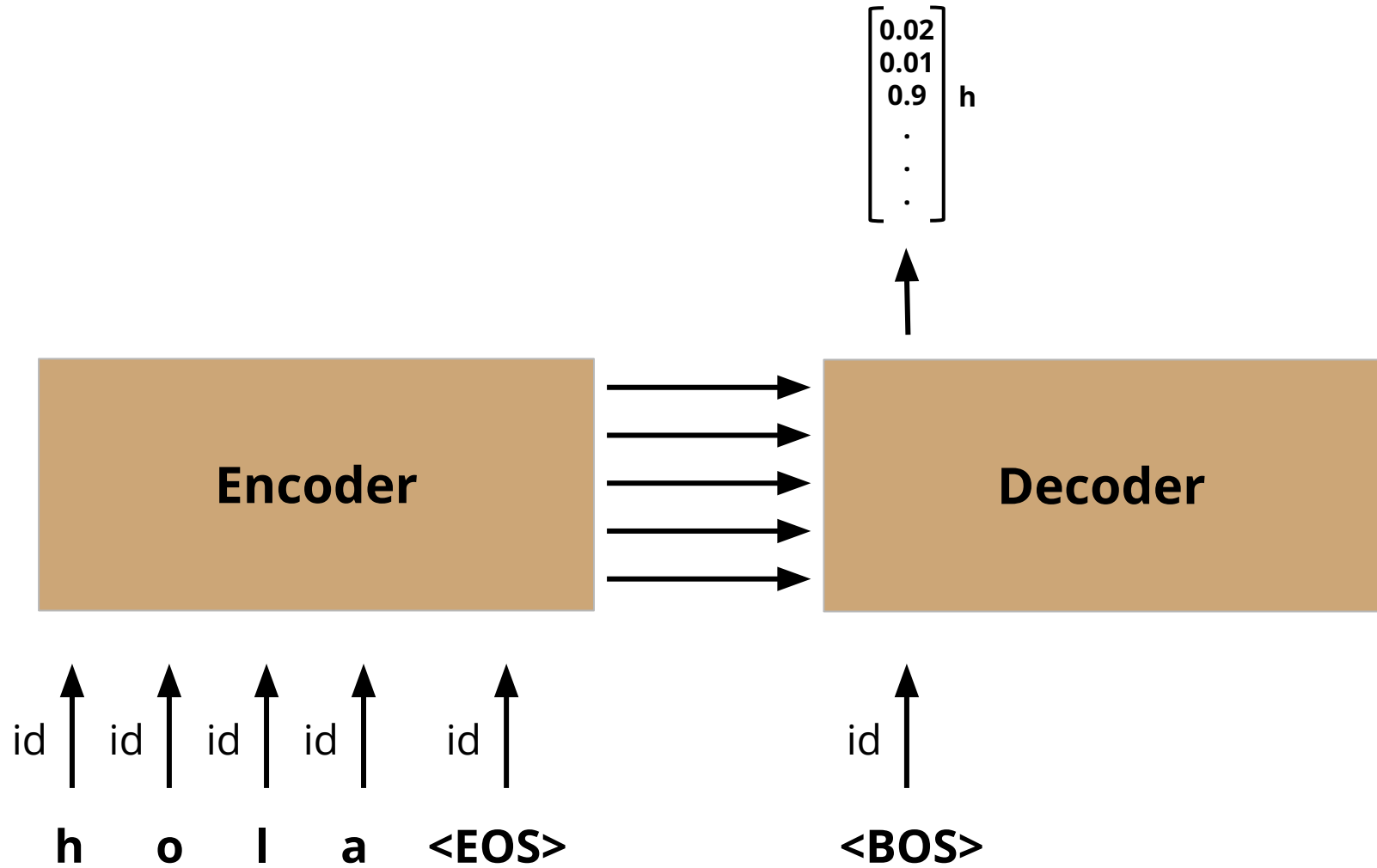
Decoder



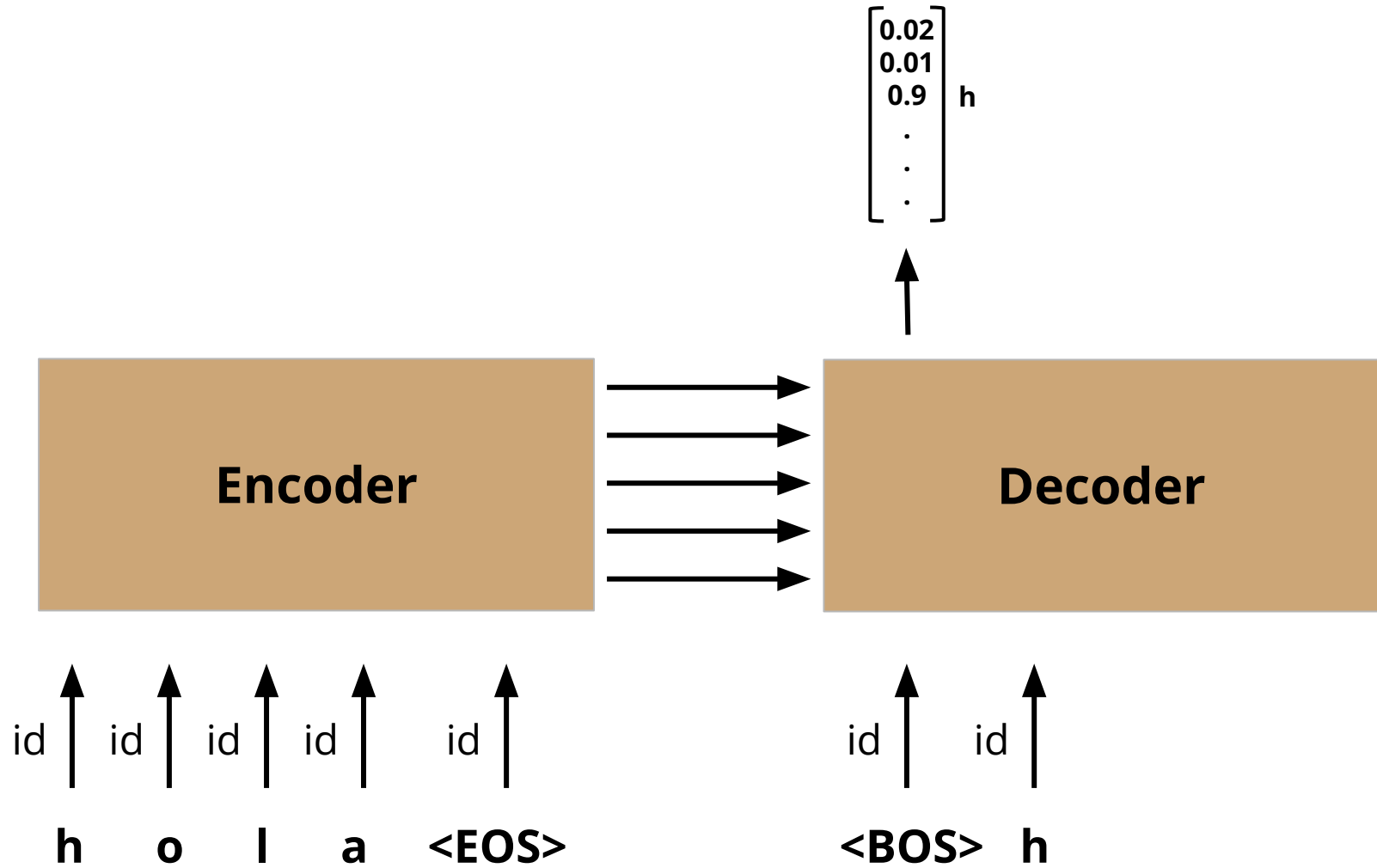
Usando el modelo...



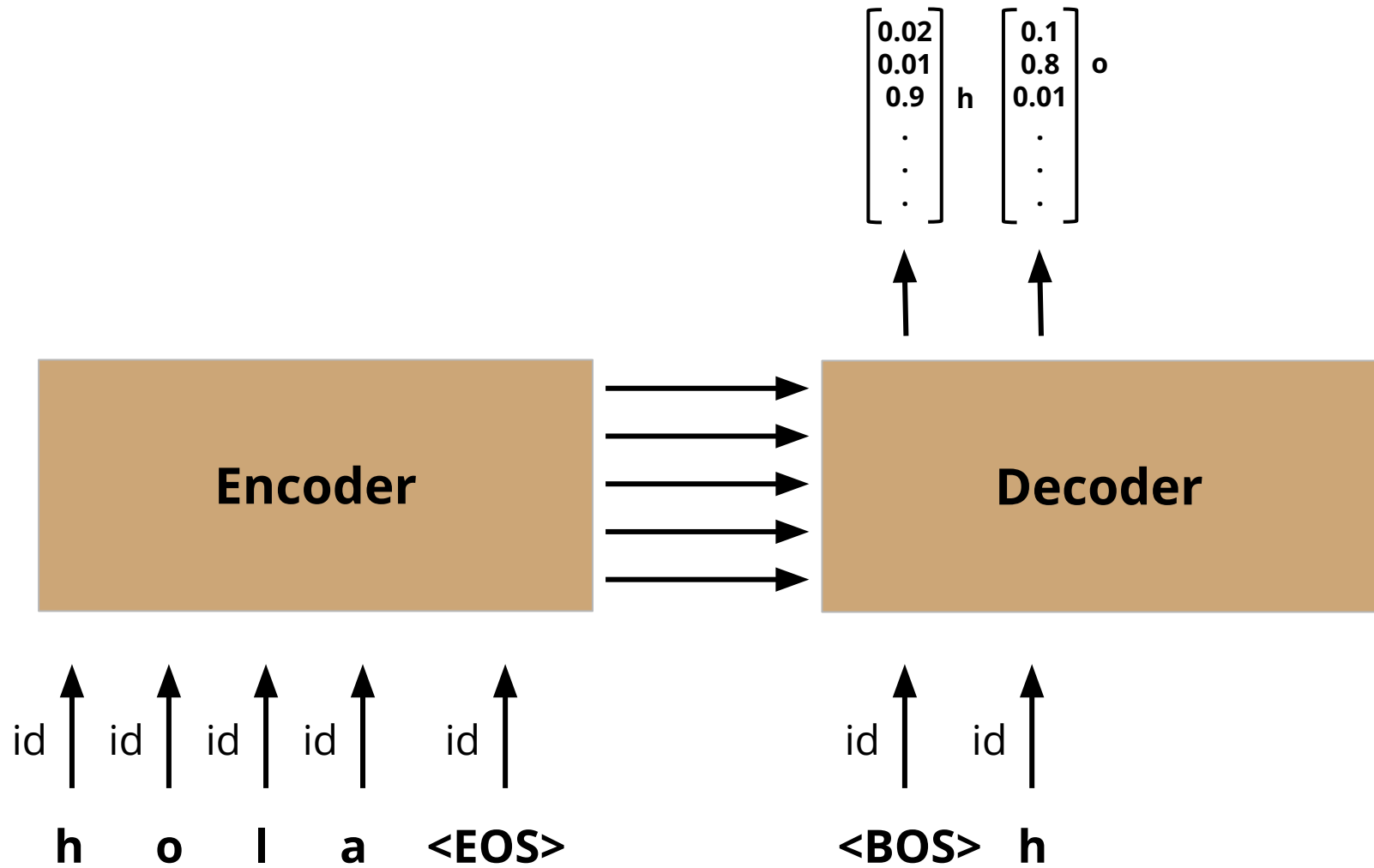
Usando el modelo...



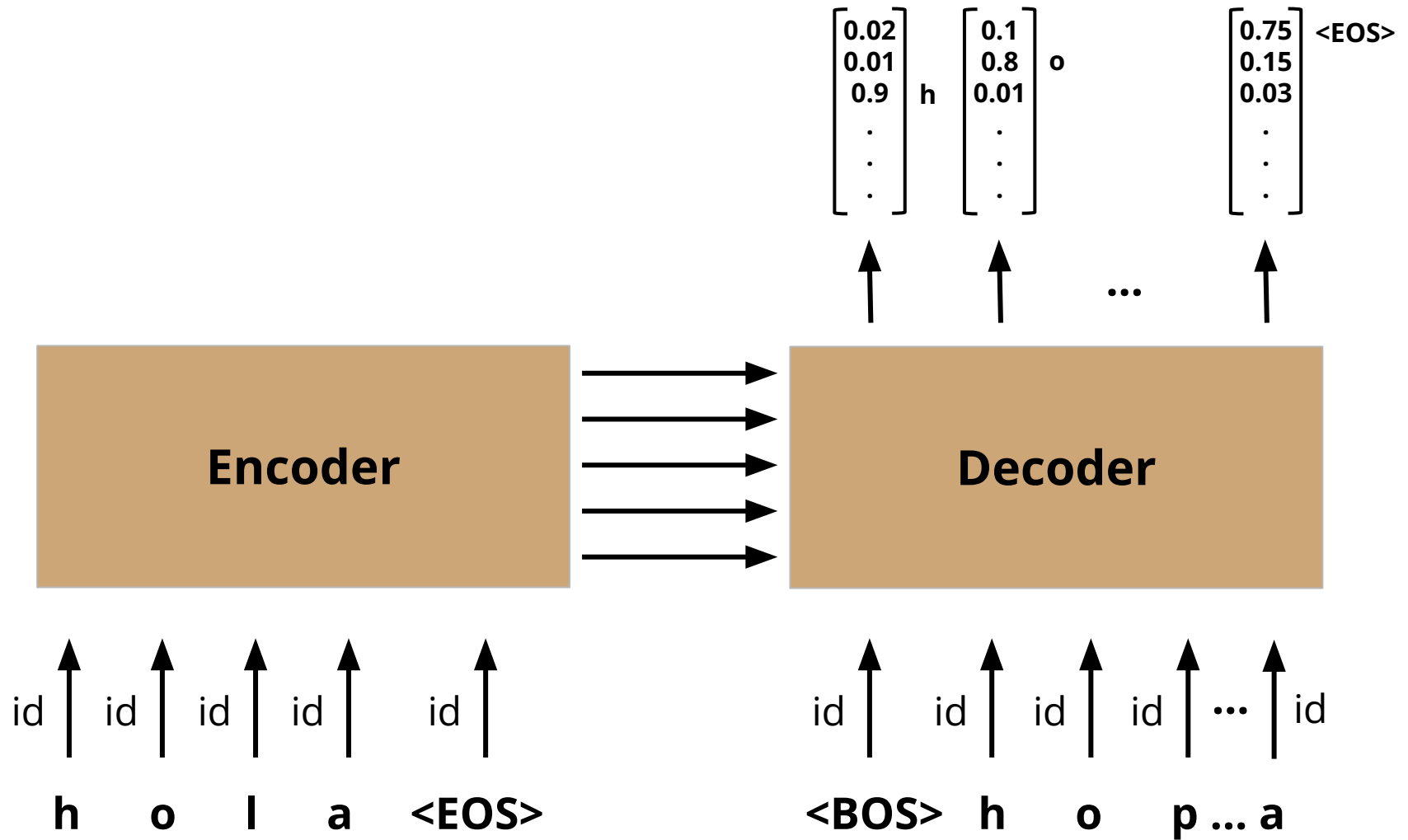
Usando el modelo...



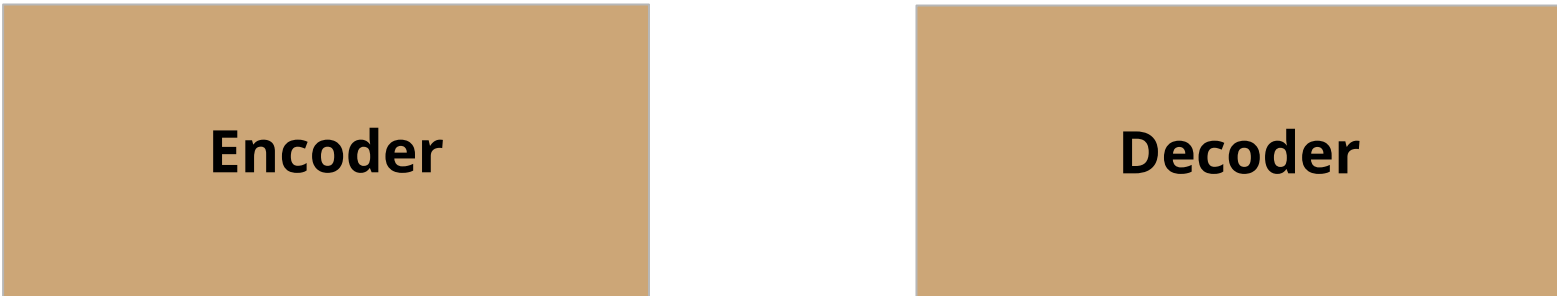
Usando el modelo...



Usando el modelo...



Entrenando el modelo...



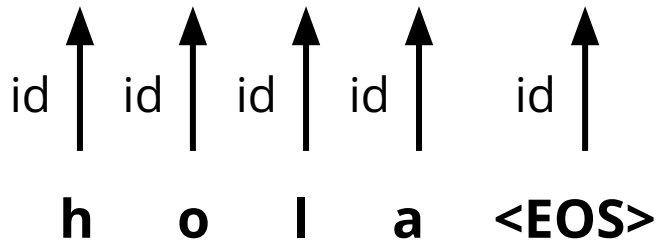
Encoder

Decoder

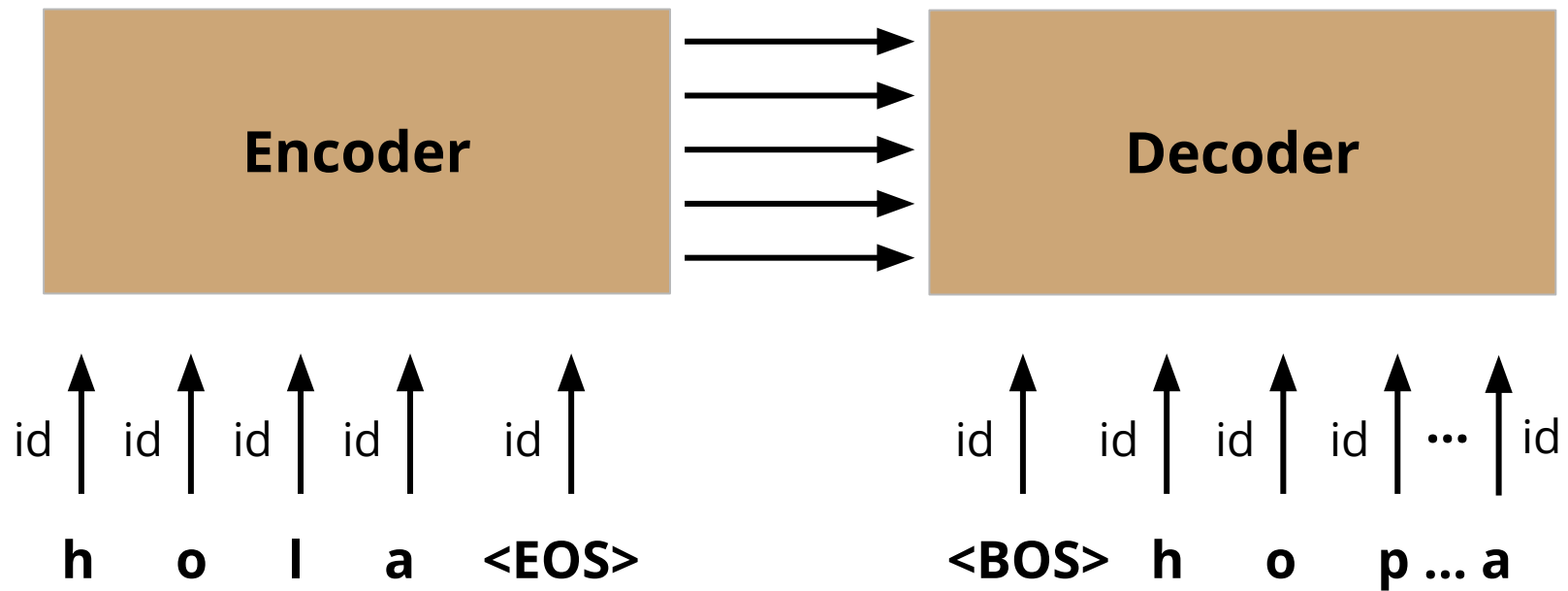
Entrenando el modelo...

Encoder

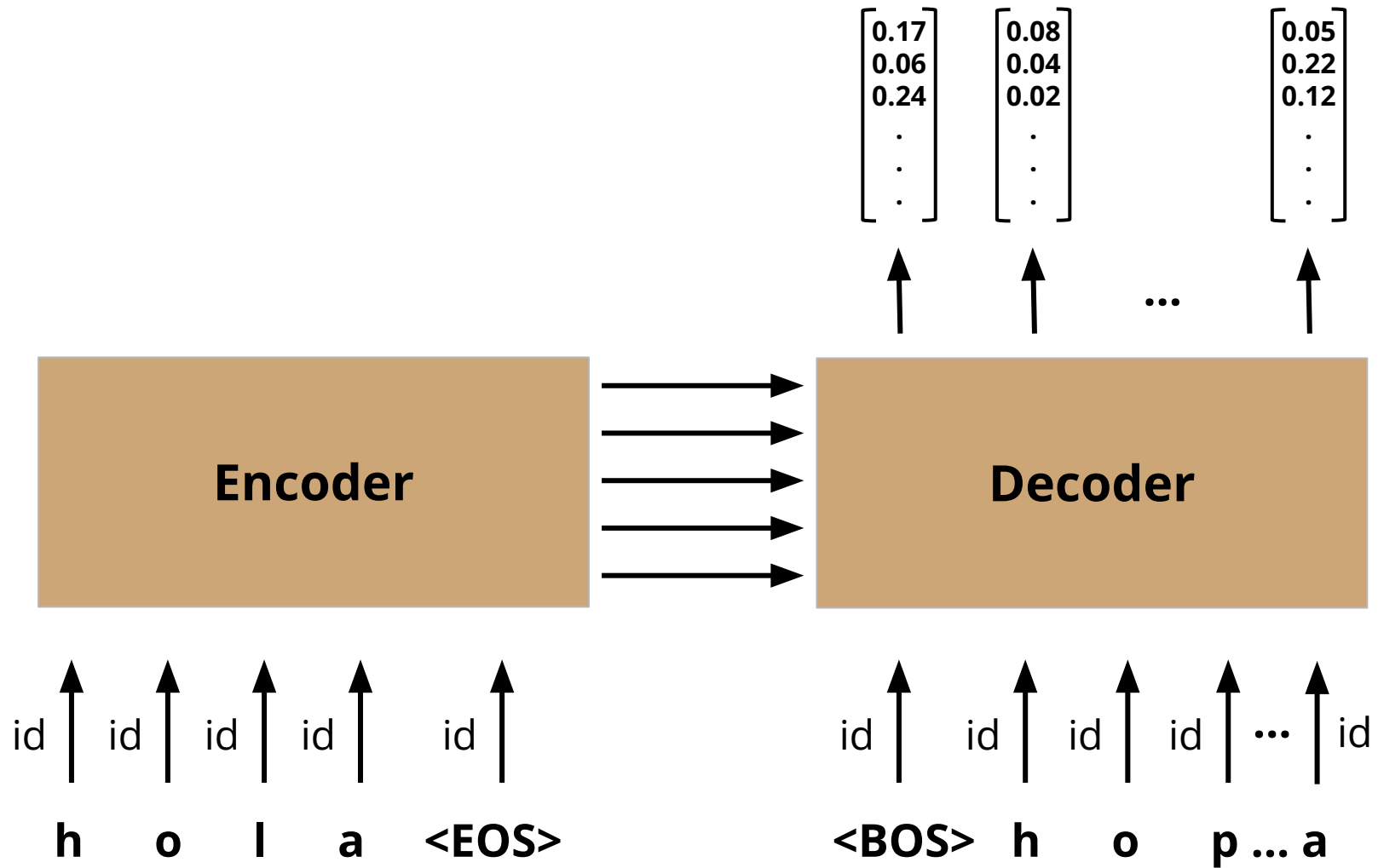
Decoder



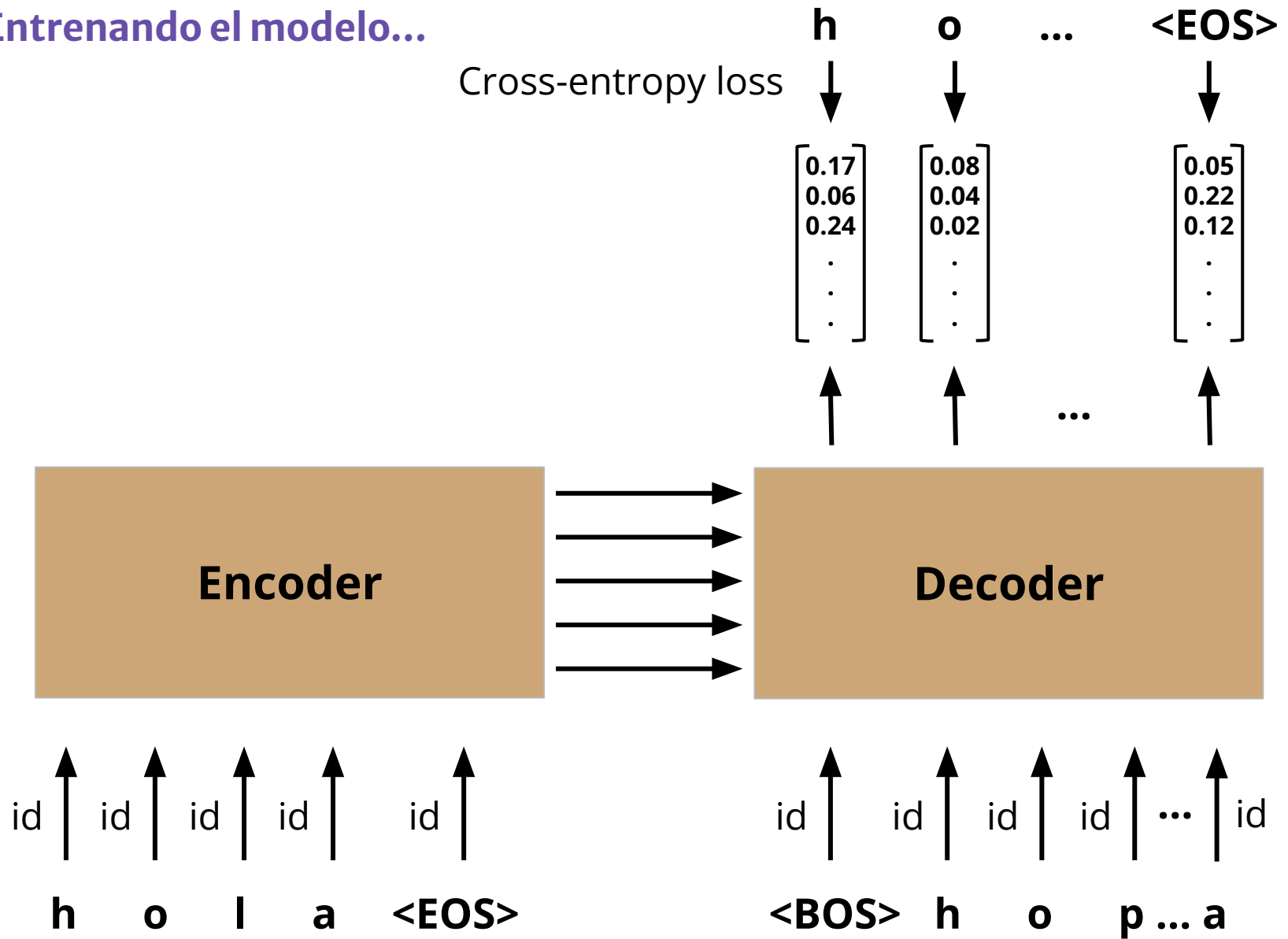
Entrenando el modelo...



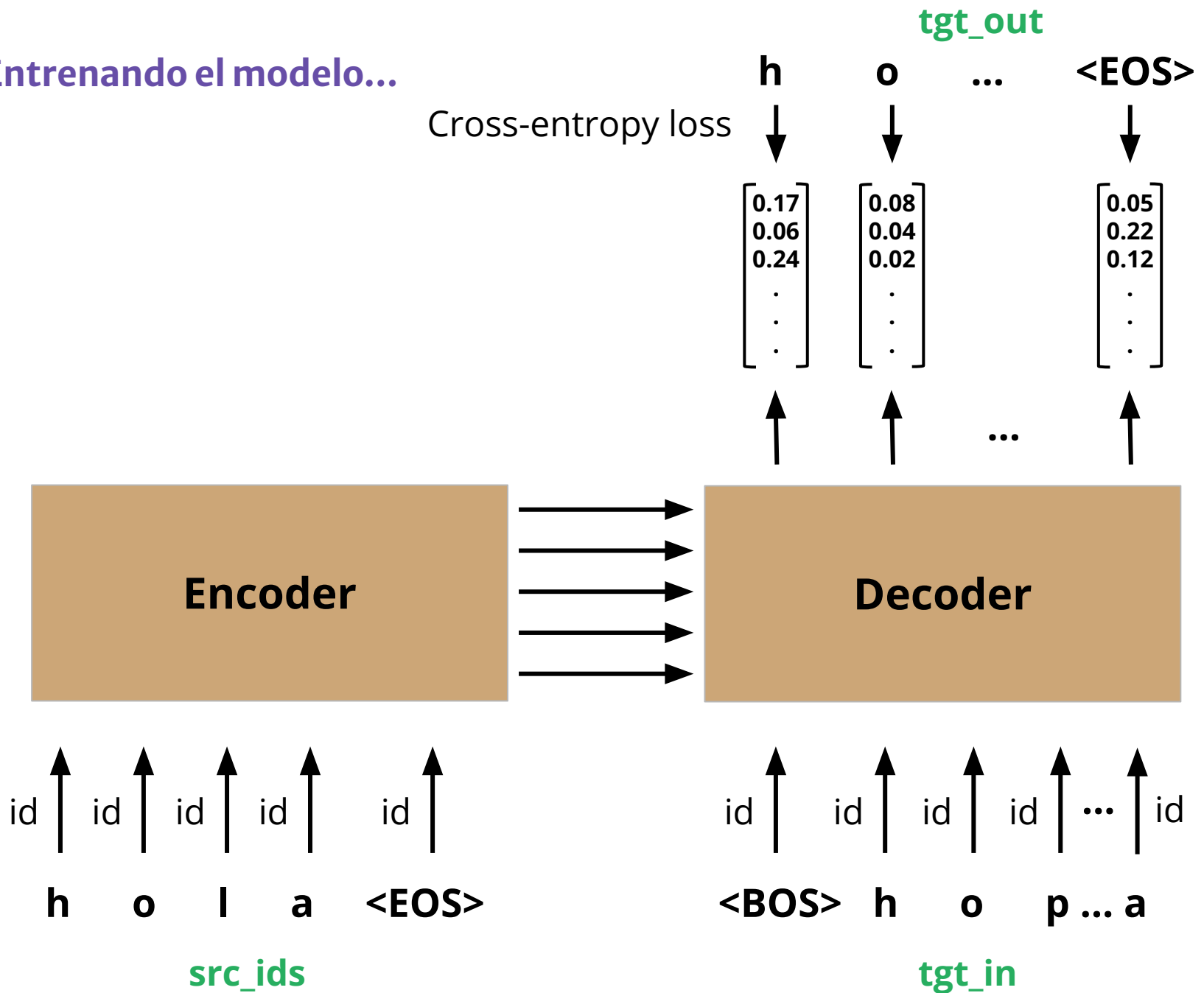
Entrenando el modelo...



Entrenando el modelo...



Entrenando el modelo...



Construyendo el minibatch...

Supongamos que el minibatch tiene las siguientes palabras:

hola<EOS>

pez<EOS>

perro<EOS>

Construyendo el minibatch...

Supongamos que el minibatch tiene las siguientes palabras:

hola<EOS> $\xrightarrow{\text{src_ids}}$ **[11, 19, 15, 4, 2]**

pez<EOS> $\xrightarrow{\text{src_ids}}$ **[20, 8, 30, 2]**

perro<EOS> $\xrightarrow{\text{src_ids}}$ **[20, 8, 22, 22, 19, 2]**

Construyendo el minibatch...

Supongamos que el minibatch tiene las siguientes palabras:

hola<EOS> $\xrightarrow{\text{src_ids}}$ **[11, 19, 15, 4, 2]**

pez<EOS> $\xrightarrow{\text{src_ids}}$ **[20, 8, 30, 2]**

perro<EOS> $\xrightarrow{\text{src_ids}}$ **[20, 8, 22, 22, 19, 2]**

Necesitamos una matriz que contenga todos los ejemplos, uno por fila:

Construyendo el minibatch...

Supongamos que el minibatch tiene las siguientes palabras:

hola<EOS> $\xrightarrow{\text{src_ids}}$ **[11, 19, 15, 4, 2]**

pez<EOS> $\xrightarrow{\text{src_ids}}$ **[20, 8, 30, 2]**

perro<EOS> $\xrightarrow{\text{src_ids}}$ **[20, 8, 22, 22, 19, 2]**

Necesitamos una matriz que contenga todos los ejemplos, uno por fila:

$$\begin{bmatrix} 11 & 19 & 15 & 4 & 2 & ? \\ 22 & 8 & 30 & 2 & ? & ? \\ 20 & 8 & 22 & 22 & 19 & 2 \end{bmatrix}$$

Construyendo el minibatch...

Supongamos que el minibatch tiene las siguientes palabras:

hola<EOS> $\xrightarrow{\text{src_ids}}$ **[11, 19, 15, 4, 2]**

pez<EOS> $\xrightarrow{\text{src_ids}}$ **[20, 8, 30, 2]**

perro<EOS> $\xrightarrow{\text{src_ids}}$ **[20, 8, 22, 22, 19, 2]**

Necesitamos una matriz que contenga todos los ejemplos, uno por fila:

$$\begin{bmatrix} 11 & 19 & 15 & 4 & 2 & ? \\ 22 & 8 & 30 & 2 & ? & ? \\ 20 & 8 & 22 & 22 & 19 & 2 \end{bmatrix} \xrightarrow{\text{usamos <PAD>}} \begin{bmatrix} 11 & 19 & 15 & 4 & 2 & 0 \\ 22 & 8 & 30 & 2 & 0 & 0 \\ 20 & 8 & 22 & 22 & 19 & 2 \end{bmatrix}$$

Construyendo el minibatch...

Supongamos que el minibatch tiene las siguientes palabras:

hola<EOS> $\xrightarrow{\text{src_ids}}$ **[11, 19, 15, 4, 2]**

pez<EOS> $\xrightarrow{\text{src_ids}}$ **[20, 8, 30, 2]**

perro<EOS> $\xrightarrow{\text{src_ids}}$ **[20, 8, 22, 22, 19, 2]**

Necesitamos una matriz que contenga todos los ejemplos, uno por fila:

$$\begin{bmatrix} 11 & 19 & 15 & 4 & 2 & ? \\ 22 & 8 & 30 & 2 & ? & ? \\ 20 & 8 & 22 & 22 & 19 & 2 \end{bmatrix} \xrightarrow{\text{usamos <PAD>}} \begin{bmatrix} 11 & 19 & 15 & 4 & 2 & 0 \\ 22 & 8 & 30 & 2 & 0 & 0 \\ 20 & 8 & 22 & 22 & 19 & 2 \end{bmatrix}$$

Lo mismo para tgt_in y tgt_out